关闭 DSH Superman 扩展套件 v1 局限：端侧 OCR、BM25 回忆与记忆压缩的实测验证

作者： Anstar, Z.[1]*、Zeng, P.[1]、DeepSeek Harness Contributors[2]

[1] 独立研究者。[2] DeepSeek Harness 贡献者社区。

*通讯作者：Anstar, Z.（anstar@…）。

摘要

智能体编码框架的记忆通常止步于单次会话。我们此前为 DeepSeek Harness 构建的 Superman 扩展套件补上了这一缺口，并如实列出八条局限。本文报告随后的「关闭局限」工作：六条已关闭，一条部分关闭，一条未关闭。扫描件经本机 macOS Vision OCR 恢复为可回忆文本，一页文本层为 0 字符的栅格化页面被识别出 3 411 个字符、其中 979 个汉字。回忆打分改为 BM25（k1 = 1.2、b = 0.75）。每轮对话另写一条上限 7 900 字符的可还原载荷，记忆体在启动时把会话合并回列表。历史图片凭记忆条目 id 取回原始字节。记忆体加入 2 000 条与 512 MiB 双上限、最旧优先淘汰、启动对账与 prune。面板上传与 Harness 捕获的 PDF 在入库时自行抽取文本。离线检查由 284 项增至 449 项，代码由 7 187 行增至 8 869 行。DOM 测试与语义回忆未完成，本文说明其边界。

关键词： 智能体编码框架；长期记忆；光学字符识别；BM25；记忆压缩；插件扩展；离线验证

1 引言

智能体编码框架把语言模型放进一个带工具循环的会话里。用户描述任务，框架读写文件、执行命令、迭代修改代码，界面因此呈「会话流 + 工具调用流」的形态。DeepSeek Harness（下称 DSH）[1] 是这类框架的一个实现：它以浏览器应用为界面，扩展点由 Cordis 插件框架 [2] 提供，界面以 React [3] 组件组装。其扩展机制允许第三方在**不修改宿主源码、不派生宿主**的前提下增加界面与路由，这既降低了分发成本，也把「能做什么」严格限制在插件作者可支配的那部分运行环境里。

我们此前报告了一套为该框架构建的扩展套件，包含三个子系统：Chat 模式提供一个不携带工具的纯对话面板；Superman 记忆体把两个模式上传的文件与产生的对话、任务记录归类进一个智能文件夹，并在每次组装系统提示词时注入记忆摘要；文献捕获在 arXiv 检索结果上增加「收藏到记忆体」按钮。该版本由 6 个 JavaScript 文件、7 187 行代码构成，由 5 套共 284 项离线检查约束，在 14 个真实 PDF 上抽出 199 942 个字符、其中 67 235 个汉字、覆盖 138 页，替换字符与 C1 控制字符为 0。

该版本在发表时列出八条局限。这一次，我们把「关闭局限」本身当作一项独立的工作来报告，理由有三。

这些子系统各自要对抗一类失效，而失效的形态决定了修复的形态。Chat 模式不携带任何工具，回答由 `text/event-stream` 帧流式返回（分帧方式即 HTML 标准中服务端发送事件的分帧方式 [4]），公式在浏览器内以 MathML [5] 排版，不引入 KaTeX [6] 或 MathJax [7] 这类完整引擎，字体回退链以 STIX 系列 [8] 为主，字符与编码的语义以 Unicode 标准 [9] 与 WHATWG 编码标准 [10] 为准。因为不能 import 宿主的 Client 包，PDF 文本抽取、编码识别与公式翻译都实现在 bundle 内部。记忆体的回忆是词法打分，其家族可追溯到逆文档频率 [11] 与 BM25 一类的概率检索框架 [12,13]；记忆目录是「一个 JSON 索引加一个原文件目录」，写入按先写临时文件再 rename 的方式串行且原子；系统提示词分段是一段动态摘要，每次组装提示词时重新计算。文献捕获依赖一次实测结论：arXiv 的 PDF 响应带 `access-control-allow-origin: *`，页面因此可以自行下载，下载失败则降级为一条可被回忆的元数据记录。会话历史的落点曾经是页面 origin 的 `localStorage` [14]，这一点后来被证明是三条局限的共同根源。

验证方法学决定了全部数字的含义。检查全部驱动**随包发布的真实文件**，在 Node 里运行：浏览器半由桩 `window.__ModuleLoader__` 装载、由桩 React 直接调用函数组件，配内存 `localStorage` 与被替换的 `fetch`；Host 半在伪 Cordis 上下文与临时记忆目录里运行。真实的部分是插件源码、契约校验、状态机、PDF 抽取与公式生成、编码探测、记忆索引与文件、SSE 重组、URL 与请求体以及 DOM 结构；替身与真实的分界在方法一节逐条列出。这套方法学在 v1 之后留下三个未回答的问题，本文逐一处理：扫描件与文字转曲文档能否恢复为可回忆文本；词法排序能否在文档长度差异悬殊时保持稳定；以及离线套件之外能否补上真实 DOM 装配的检查。

其一，**局限是设计契约的推论，不是疏忽**。八条局限中有五条直接来自本项目被迫接受的约束：浏览器半不能 import 宿主的 Client 包，所以 PDF 阅读器、公式渲染器与编码探测器都必须自行实现，其中 PDF 阅读器只能是文字层读取器（PDF 的对象模型与流过滤器由 ISO 32000-1 [15] 与 Adobe 的 PDF Reference [16] 规定）；通信只能走已鉴权的同源 `/api` Fetch 路由，所以没有可用于编排跨 origin 状态的服务端；记忆是「一个 JSON 索引加一个原文件目录」，所以它需要一个容量策略而不只是条数上限；开发环境没有可编程的浏览器控制，所以端到端 DOM 验证缺位。逐条修复这些局限，等于逐条检验当初的设计取舍是否仍然成立。

其二，**修复必须留下可复核的证据**。一条局限是否关闭，不能由作者宣布。我们在每条修复上都规定了可测量的判据：扫描件的文本层字符数与 OCR 字符数、稀有词文档在排序中的位次、空 localStorage 的全新 origin 能否恢复会话、带记忆条目 id 的图片请求体里有没有 base64 字节、条数上限触发时文件与索引项是否同时消失、两端 PDF 抽取器在真实文件上的

文本是否逐字相同。这些判据全部落在离线检查套件里，因此「关闭」是可重复的实验结论，而不是验收意见。把判据写进检查的同一条纪律，也要求我们预先说明每条检查的能力边界：整套验证没有浏览器，浏览器半以**桩模块加载器 + 桩** React 的方式在 Node 里运行，函数组件被直接调用而不是渲染，`document`、`navigator`、计时器、`localStorage` 与 `fetch` 都是替身，因此它能证明通信形状、契约校验、状态机与解析结果，不能证明像素级外观、真实字体回退与真实鉴权链路 [14]。

其三，**诚实的不完整必须与修复同等显眼**。八条中有一条只做到部分关闭，一条完全没有关闭。把它们写成已完成，会让整篇报告失去意义；把它们删掉，则掩盖了本工作最重要的方法论主张——离线套件能证明什么、不能证明什么。

为什么局限值得一篇独立的报告？因为在智能体框架里，**上下文的管理方式决定能力上限**。研究表明，长上下文并不等于有效上下文：信息在长窗口中的位置显著影响模型取用 [17]；把上下文当作可管理的操作系统资源 [18]、让行为体自己积累与检索经验 [19]、把推理与动作交错 [20]、让模型自行决定调用外部工具 [21]，都已经是框架层的常规设计。记忆体正是这些设计共同依赖的那一层基础设施，而它的质量由两个可测量的量决定：**取回得准不准**与**留得住多久**。

检索这一侧有清晰的技术谱系。词法检索的经典形式建立在词项特异性之上 [11]：稀疏词项的逆文档频率（IDF）衡量它区分文档的能力，Okapi 的 BM25 把这一思想与词频饱和、文档长度归一化结合 [12,13]，Salton 与 Buckley 对词项加权方法作了系统比较 [22]。语义侧则依赖嵌入表示：把文本编码成向量后做稠密检索 [23]、句向量编码 [24]、晚期交互 [25]、生成模型与检索的耦合 [26]、十亿级相似度搜索 [27] 与近似最近邻图索引 [28]，都是这一路线的成熟形态；其表示能力来自 Transformer 架构 [29] 与在大规模语料上预训练的模型 [30,31]。两条路线的取舍很清楚：词法检索确定、便宜、可离线验证，语义检索覆盖面更广但需要一个可用的嵌入或向量服务。v1 的记忆体选择了前者并如实记录了它的缺口；v2 把这套打分升级为 BM25（2.2），但没有改变这一选择的性质。

本文的贡献有四点。

贡献一：一套端侧的扫描件恢复路径，并给出逐例测量。 我们先用 PDFKit [32] 把页面栅格化、再用 macOS Vision 的 `VNRecognizeTextRequest` [33] 识别中文与英文，并把这条路径以「可选 bundle + 浏览器半自动降级」的方式接进 Chat 上传流程与记忆体收录流程。测量给出三类真实情形的对照：文本层为 0 字符的栅格化页面被识别出 3 411 个字符（979 个汉字）；第二个栅格化页面文本层仅 1 个字符，OCR 给出 1 341 个字符（1 072 个汉字）；一份 19 页、内嵌 CMap 损坏的招贴抽出 9 423 个字符而其中仅 9 个汉字，OCR 给出 11 333 个字符（66 个汉字）。第三例说明字符数不是唯一判据：字符多不等于读得对。

贡献二：把回忆打分替换为按其原始定义实现的 BM25。 原先的加法词法打分（名称命中加权、正文命中加权，加一个上限 1.5 的新鲜度项）在文档长度差异大时偏向长文档，且常见词与稀有词同权。现在回忆改为 BM25 [12,13]：文档频率按整库现算，IDF 采用 `log(1 + (N − df + 0.5)/(df + 0.5))`，饱和常数 $k1 = 1.2$、长度归一化 $b = 0.75$，字段按名称加权 3、正文加权 1 合并为带权词频与带权长度，最后叠加新鲜度项。返回结构与摘要字符串不变，因此调用方无需改动。

贡献三：让对话与图片脱离浏览器 origin。 每一轮结束除写一条散文式记录外，再写一条 `chatstate:<id>` 记录，内容是上限 7 900 字符的可还原 JSON；面板启动时先用本地缓存在同步路径上渲染，随后把记忆体里的会话合并进来。历史图片在存储时记下记忆条目的 id，发送前通过 `GET /api/superman/file?id=…` 取回原始字节。记忆体自身则加入 2 000 条与 512 MiB 两个上限、最旧优先淘汰、启动对账、手动 `prune` 与淘汰计数。

贡献四：入库即抽取，以及一套从 284 **项增长到** 449 **项的离线验证方法学。** 我们把浏览器半的 PDF 抽取器移植成 Node 模块 `superman/pdf.js`（799 行），使面板上传与 Harness 捕获的 PDF 在入库时就带文本，并用「两端在同一批真实文件上必须给出逐字相同的文本」这一检查约束移植的正确性。全部离线检查在一个伪 Cordis 上下文、桩模块加载器、桩 React、内存 `localStorage` 与被替换的 `fetch` 上驱动**随包发布的真实文件**；我们同时明确写出它证明不了什么。

本文余下部分这样组织。第 2 节给出结果，按 v1 的八条局限 L1–L8 分小节，每小节给出问题、方法、测量与图表引用。第 3 节讨论这次「关闭局限」的实践教益，包括效度威胁，并如实说明 L7 只做到部分关闭、L8 完全未关闭。第 4 节给出方法：插件模型、OCR 流水线、BM25 的完整公式、存储模式与原子性、压缩策略、DOM 工具链与复现命令。附录 A–E 给出 API 动作清单、验证套件清单、语料与 OCR 测量表、局限到修复到证据的登记表，以及复现命令与仓库布局。图 1 对照两版的整体拓扑：(a) 是 v1，(b) 是 v2，新增的部分是 OCR 路径、会话水合与压缩策略。

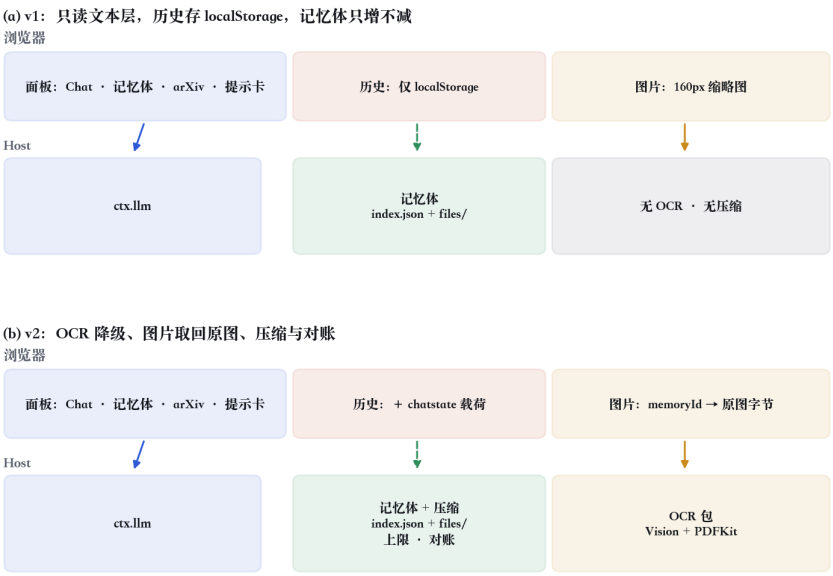


图 1. 扩展拓扑的 v1 与 v2 对照：(a) v1；(b) v2，新增 OCR、会话水合与压缩。

2 结果

本节按八条局限分小节。每条小节的结构相同：先复述 v1 的问题，再说明 v2 的修复方式，最后给出测量、证据与图表引用。八条局限的状态与各自的检查增量汇总在图 2 与表 4；两版的整体拓扑对照见图 1。

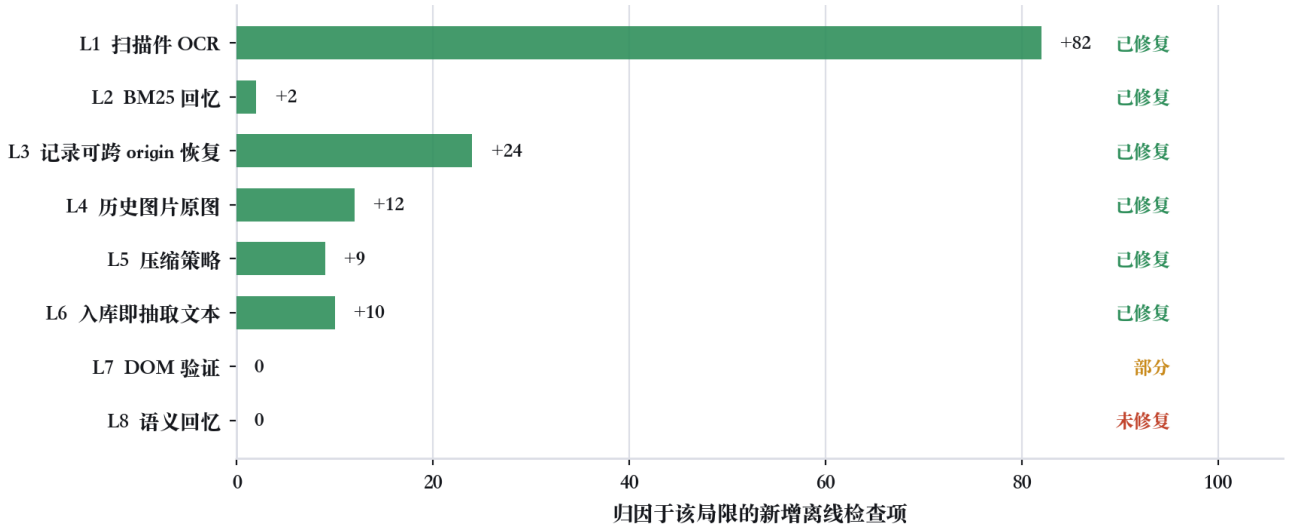


图 2. 八条 v1 局限的修复矩阵：已关闭、部分关闭与未关闭，并标注每条局限对应的检查项增量。

表 2. 局限到修复到证据到状态的登记表。

#	v1 局限	v2 修复	证据	状态
L1	PDF extraction reads the text layer only; scans yield nothing	on-device OCR through macOS Vision, wired into the Chat upload path as an automatic fallback		已修复
L2	recall was a flat additive lexical score that favoured long documents	BM25 (k1 = 1.2, b = 0.75) with IDF over name/body/record fields plus the recency term		已修复
L3	conversation history lived in the page origin's localStorage	every finished turn also writes a restorable JSON payload (chatstate:<id>, ≤ 7 900 characters) that merges on start-up		已修复
L4	images in history kept only a 160 px thumbnail and resent as unavailable	an image remembers the memory item id of its bytes; a resend fetches the original from the file route		已修复
L5	the memory grew without a compaction policy	maxItems 2 000 and maxBytes 512 MiB caps, oldest-first eviction, start-up reconciliation, prune, eviction counters		已修复
L6	panel uploads and Harness captures stored PDF bytes without text	superman/pdf.js, a Node port of the browser extractor, parses on store (200-page / 400 000-character caps)		已修复

#	v1 局限	v2 修复	证据	状态
L7	no automated browser-level (DOM) end-to-end test	a workspace-local jsdom + React toolchain and tools/dom-check.mjs mount the panels with createRoot; environment differences remain		部分修复
L8	recall is not semantic	no embedding service exists on this machine; retrieval stays lexical (now BM25), the seam is recall(q)		未修复

2.1 L1：扫描件与文字转曲文档通过端侧 OCR 恢复为可回忆文本

问题。 v1 的 PDF 读取器只读文字层。对象扫描、流解码、`/ToUnicode` CMap 还原、内容流重放与页面装配构成了完整的抽取流水线，但流水线的起点是内容流里的文本算符：一页被栅格化成图像、字号很小的矢量轮廓、或者内容流中根本没有 `Tj`／`TJ` 的文档，读出来就是零个字符。v1 对此的处理是诚实的——报告 `no-text` 或 `unreadable`，而不是输出替换字符——但诚实不等于可用：一份扫描合同在记忆体里只是一个带文件名的附件，回忆只能命中它的名字，正文一个字也检索不到。v1 语料中 14 份文件有 9 份落入这一分支。

修复。 我们为这一类文档补了一条本机 OCR 路径，形态是「可选 bundle + 自动降级」，而不是把 OCR 塞进 PDF 读取器。`tools/ocr-pdf.swift` 是一个约 210 行的自包含 Swift 程序：用 PDFKit [32] 打开文档并逐页渲染为位图，再用 Vision 框架的 `VNRecognizeTextRequest` [33] 在位图上识别文本。识别请求的配置是 `recognitionLevel = .accurate`、`recognitionLanguages = ["zh-Hans", "en-US"]`、`usesLanguageCorrection = true`；为了不把超大字号的页面渲染成一张耗尽内存的位图，最长边被限制在 4 000 像素，超过时按比例缩小。每个页面识别完成后立刻写出 `## page N` 标记与该页的文本行，因此即使调用方在超时后杀掉进程，已完成的页面仍然可用。

`superman-ocr` 是承载它的 bundle：Host 半注册唯一一条路由 `POST /api/superman-ocr/api`，动作只有 `health` 与 `ocr`；它把请求中的 base64 载荷校验为 `%PDF-` 开头的 PDF，写入工作目录下的临时文件，然后用 `swiftc -O` 把 Swift 工具编译到工作目录并在**每个进程内只编译一次**，并发调用共享同一个进行中的编译 Promise；编译产物按 mtime 与源文件比较后缓存复用，临时文件在 `finally` 中删除。Chat 半的策略是静默降级：抽取结果为空时才调用 OCR 路由，识别成功则把文本作为低可信度正文内联进上下文，并在正文前加一句「以下内容由本机 OCR 识别，可能有错字」，同时连带文本存入记忆体；`superman-ocr` 未安装或调用失败时保留原来的「无文本层」提示，不报错、不阻塞发送。

测量。 三类真实情形被逐例记录，见表 3 与图 7。

表 5. OCR 测量：每个案例的文本层字符数与 OCR 字符数／汉字数、页数与耗时。

用例	页	文本层字符	文本层汉字	OCR 字符	OCR 汉字	毫秒
光栅化页面 A	1	0	0	3,411	979	—
光栅化页面 B	1	1	0	1,341	1,072	—
CMap 损坏的 19 页册子	19	9,423	9	11,333	66	—
光栅化页面 A（拉丁）	1	0	0	2,056	0	1,470
光栅化页面 B（汉字）	1	0	0	1,376	992	880

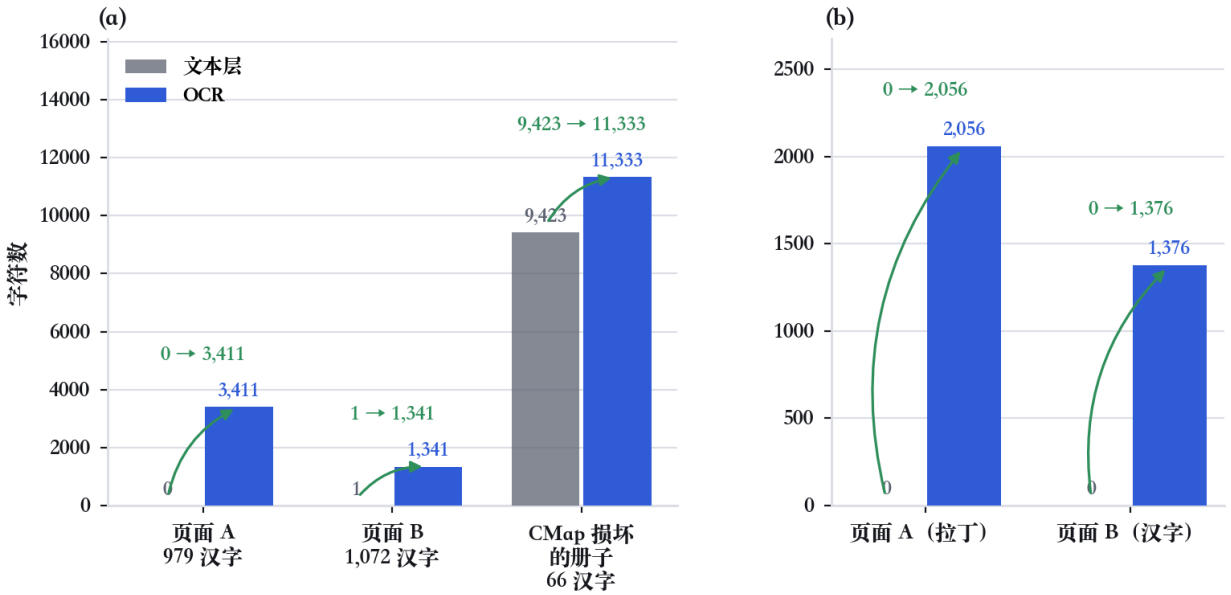


图 5. 栅格化与 CMap 损坏页面上文本层与 OCR 的产出对比，标注「0 → 3 411」。

第一例是决定性的：一页栅格化成纯图像 PDF 后，文本层含 0 个字符；OCR 在同一页上读出 3 411 个字符，其中 979 个是汉字。文本层与 OCR 的差距不是百分比，而是「零」与「可读文本」的区别，这正是本地记忆从「只记得文件名」变为「记得内容」的门槛。第二例把这一差距放在接近对照的条件下：另一页栅格化后文本层只剩 1 个字符，OCR 读出 1 341 个

字符，其中 1 072 个汉字。第三例最值得注意：一份 19 页的招贴内嵌 CMap 已损坏，文字层抽取给出 9 423 个字符，但其中只有 9 个汉字——也就是说，抽取器成功读出了大量字节，却几乎完全误解了它们；OCR 在同一份文档上给出 11 333 个字符、66 个汉字。这一例说明字符数不是唯一判据：**字符多不等于读得对**，因此 OCR 结果在正文里带有明确的可信度标注，而不是被当作与文字层等价的文本。三类情形的测量与生成这些对照的语料无关，语料本身见附录 C；其中作为记忆体收录样本的一篇 arXiv 预印本仅用作语料样本，不承载任何设计主张 [34]。

误差与边界。 OCR 是识别，不是拷贝，因此它引入的误差需要被讨论而不是被掩盖。Vision 与 Tesseract [35] 一类引擎都把文字识别当作序列解码问题，其经典形式与隐马尔可夫模型上的解码相关 [36]；在字词层面，识别误差通常用编辑距离衡量 [37]。在我们的样本里，可观察到的误差形态有三类：行内空格被插入或吞掉，字形相近的汉字被替换，以及标点被规范化为全角或半角。这三类误差对「回忆」这一用途的伤害远小于对「引用」的伤害，因此我们把 OCR 文本定位为可检索、可提示的正文预览，而不是可逐字引用的来源。上限方面：一次调用最多 200 页，默认 40 页；载荷不超过 32 MiB；返回文本截断到 400 000 字符；单次调用的时间预算 600 000 ms，首次编译的预算 300 000 ms；渲染比例被限制在 0.5–6 之间，默认 2。这些数字全部有对应的离线检查（见 2.9 与附录 B）。

在整体流程里的位置。 OCR 不是一条独立的功能，而是记忆体入库流水线的一个可选环节：条目入库 → 分类（PDF／图片／程序／其他）→ 抽取正文（PDF 走文字层读取器）→ 文字层为空时尝试 OCR → 建索引（正文进入条目的可见字段）→ 回忆（BM25）→ 压缩（上限、淘汰、对账）。图 4 画出这条流水线，并标注两个容量上限与启动对账的位置。把 OCR 放在「抽取之后、建索引之前」而不是放在回忆时，意味着识别成本只在入库时支付一次，之后的每一次回忆都受益；代价是入库变慢，且识别质量在那一刻就被固定下来。



图 9. 记忆体生命周期：入库、分类、抽取或 OCR、建索引、回忆、压缩，并标注两个上限与启动对账。

2.2 L2：回忆打分由加法词法打分改为 BM25

问题。 v1 的回忆是一个扁平的加法打分：名称命中每个词元加 4 分，正文命中加 1.5 分（词元长度 ≥ 2）或 0.6 分，同一词元在正文出现超过 3 次再补 0.5 分，最后加一个上限 1.5 的新鲜度项。这个打分有两个结构性缺陷。第一，它**没有长度归一化**：一份 90 000 字符的招生简章命中 30 个词元就一定压过一份 800 字符的笔记命中 28 个词元，尽管后者几乎句句相关。第二，它**不区分词的稀有程度**：在每份文档里都出现的「的」「the」「报告」与只在两份文档里出现的专有名词得到同样的权重，因此常见词会淹没真正的检索信号。缺陷在文档长度差异大的记忆体里被放大——而记忆体恰恰经常同时存放 5 MB 的报告与 2 KB 的代码片段。

修复。 回忆改为 BM25 [12,13,22]，参数 k1 = 1.2、b = 0.75，文档频率按整库现算，IDF 采用 `log(1 + (N − df + 0.5)/(df + 0.5))`；名称（或记录标题）按权重 3、正文按权重 1 合并为带权词频与带权长度；每个词元的贡献是 `IDF·f·(k1 + 1)/(f + k1·(1 − b + b·L/L■))`，其中 f 是带权词频、L 是本文档带权长度、L̄ 是整库平均带权长度。全部词元的贡献求和后，再加一个上限 1.5 的新鲜度项。这一段与 v1 的差别是可替换的最小面：`recall(q)` 的函数名、返回结构与渲染出的摘要字符串完全不变，因此两个调用方（Chat 面板发送前的回忆注入、Harness 提示词摘要）不需要任何改动。

测量。 我们用一个两文档的示例把两个打分函数的差别算清楚；该示例画在图 9，下面先把数算完。文档 A 的正文是「记忆体 完成 的 对账 与 淘汰 等 操作」，名称是 ■■■`notes.txt`；文档 B 的正文是「记忆体 与 对话 任务 记录 提示卡 归档 对账 淘汰 回忆 收录 摘要 等 十二个词 的 清单 用于 词频 统计」，名称是 ■■■`long-notes.txt`。查询是两个词：「对账 记忆体」。在 v1 的加法打分下，A 得 4（名称：两个词元各一次）+ 2 × 1.5（正文两次命中）= 7；B 得 4（名称里含「记忆体」一次）+ 3 × 1.5（正文三次命中）+ 0.5（同一词元在正文出现超过 3 次的加成）= 9，**长文档胜**。在 BM25 下，两个文档都含这两个词元，因此「记忆体」与「对账」的文档频率都是 df = 2，`IDF(2, 2) = log(1 + 0.5/2.5) = 0.1823`，两个词元权重相同。差别全部来自长度归一化：名称权重 3、正文权重 1，于是 A 的带权长度 L = 11，B 的 L = 22，整库平均 L̄ = 16.5；同一带权词频 f = 1 在 A 中的归一化项是 `1 + 1.2 × (1 − 0.75 + 0.75 × 11/16.5) = 1.9`，在 B 中是 `2.5`，因此 A 每个词元贡献 0.2111、合计 0.4222，B 每个词元贡献 0.1604、合计 0.3209，**排序翻转**。两份文档的新鲜度项相同，不参与这一差别。这个示例说明的不是「BM25 更准」，而是 L2 的原始表述成立：没有长度归一化时，加法打分系统性地偏向长文档。

示例只是算术，真正的证据在检查套件里。记忆体 Host 检查套件中的 BM25 检查族直接构造一个三文档记忆——`rare-zebra-notes.txt`（正文 `zebra sighting`）、`common.txt`（正文由 30 次 `the` 组成）、`filler.txt`（正文 `the quick brown fox`）——然后断言四件事：查询 `zebra the` 时，含稀有词的文档排在重复常见词之前（检查「BM25 把稀有词文档排在重复常见词之上」）；只查询 `zebra` 时的最高分高于只查询 `the` 时的最高分（检查「每份文档都携带的词得分低于稀有词」）；`idf(3,3) × 5 < idf(1,3)`（检查「IDF 让在每份文档中都出现的词几乎不占权重」，实测 `idf(3,3) = 0.1335`、`idf(1,3) = 0.9808`，即 df = N 的词元权重只有稀有词元的七分之一左右）；以及 `recall('')` 返回空摘要、回忆既按名称也按正文命中（查询 `numpy` 命中一份 Python 文件）。这四条合起来覆盖了 L2 的两个缺陷：长度与词频的关系由示例与排序断言覆盖，稀有度由 IDF 断言覆盖。

接口与代价。 回忆动作的请求与响应形状未变，变化的是内部打分与统计；记忆体与 OCR 的动作清单及其 v2 变化见表 2。

表 6. 记忆体与 OCR 的 API 动作：v2 新增或变更的部分。

模块	路由	动作	状态	说明
`superman`	/api/superman/api (+ GET /api/superman/file?id=)	`list`	沿用	契约不变，stats 增加淘汰计数
`superman`	/api/superman/api (+ GET /api/superman/file?id=)	`store`	v2 变更	入库时由 Host 抽取 PDF 文本
`superman`	/api/superman/api (+ GET /api/superman/file?id=)	`note`	沿用	契约不变
`superman`	/api/superman/api (+ GET /api/superman/file?id=)	`get`	沿用	契约不变，返回抽取出的文本
`superman`	/api/superman/api (+ GET /api/superman/file?id=)	`delete`	沿用	契约不变
`superman`	/api/superman/api (+ GET /api/superman/file?id=)	`recall`	v2 变更	BM25（k1 1.2、b 0.75）加 IDF 取代加法打分
`superman`	/api/superman/api (+ GET /api/superman/file?id=)	`prune`	v2 新增	手动压缩通道，返回被清理的内容
`superman-ocr`	/api/superman-ocr/api	`health`	v2 新增	{ready, bin, compiled, version}
`superman-ocr`	/api/superman-ocr/api	`ocr`	v2 新增	{text, pages, characters}，Vision 引擎

代价有三点必须说明。其一，BM25 需要整库的文档频率，因此每次回忆都要遍历全部条目并构建文档向量；v1 的加法打分同样要遍历，但只做字符串包含判断。在 2 000 条上限的记忆体上，这是可接受的开销，但它把回忆的复杂度从「与条目数线性、常数极小」推向「与条目数线性、常数较大」。其二，中文按单字与相邻二元组切分，因此 BM25 的「词」在中文上并不等于词：这一点没有变，L8 讨论的语义缺口依旧。其三，字段权重的具体取值（名称 3、正文 1）是本实现的取舍，示例与检查都只验证方向性结论（稀有词优先、长文档不因长而胜），不声称这组权重在通用检索基准上最优。费用与收益的另一面是**可回退性**：BM25 的所有参数（k1、b、名称权重、新鲜度上限与窗口）都是常量，改动其中任何一个都不会改变 `recall` 的签名或返回结构，因此一次排序质量上的回归可以通过改回常量来撤销，而不需要回滚调用方。这一点在实践中比「BM25 是否最优」更重要：一个可以被安全调整的打分函数，比一个理论上更好但难以回退的打分函数更适合长期维护的记忆体。

2.3 L3：对话记录脱离页面 origin，可从记忆体恢复

问题。 v1 的会话历史写在**本页面** origin **的** `localStorage` [14] 里，键为 `dsh.chat-mode.history.v1`，是一份带版本号的 JSON 文档，含会话列表与每条会话的消息与模型选择；上限是最多 40 个会话、每会话 200 条消息、整列表 2 500 000 字符、每个文本附件保留 40 000 字符正文、推理轨迹 20 000 字符；写入走 600 ms 去抖，超出配额时按最旧优先丢弃会话并重试，最多 6 轮。这个选择的代价写得很清楚：清除站点数据就清空记录，而且在另一个端口上运行的 `dsh web` 会各自保存一份，两个 origin 的历史互不可见。它还有第三个、当时未被列为局限的代价：**记忆体里那条「对话记录」是散文式的**（标题 + 正文摘要），它服务于回忆检索，却不能被还原成会话。

修复。 修复的思路是让记忆体同时保存「可被检索的散文」与「可被还原的结构」两份表示。每一轮结束，面板除写原有的散文式记录外，再写一条 id 为 `chatstate:<■■id>`、`kind` 为 `conversation` 的记录，其正文是**可还原的** JSON：

```
{ v: 1, title, updatedAt, selection,
  messages: [ { role, text,
                attachments: [ { name, kind, bytes, mediaType,
                                 thumb, detail, memoryId } ] } ] }
```

文本附件的正文保留前 2 000 字符，图片附件保留它的 `memoryId`（见 2.4）。序列化时先整段序列化，若超过 7 900 字符就**丢弃最旧的轮次**再序列化，循环直到装得下；若只剩一两条消息仍然超长，才把每条消息的正文截断到 600 字符。这里的选择是刻意的：宁可丢轮次，也不产生一个语法上被截断、因而无法解析的 JSON。

读取侧的时序是「本地先渲染、记忆体后合并」。面板启动时仍**同步**用本地缓存渲染，保证首屏不空白；随后异步向记忆体 `list`，从记录中筛出 `chatstate:` 前缀的条目，逐条 `JSON.parse` 并用版本号 `v === 1` 与 `messages` 是否为数组做校验，再按 id 合并进当前列表，把结果回写本地缓存。合并规则是「新的胜过旧的」：本地没有这条会话时采纳记忆体版本（全新 origin 的恢复路径），记忆体版本的时间戳晚于本地时替换本地副本（跨端口与跨浏览器时的收敛路径），本地副本更新时保持不动（避免用自己的旧副本覆盖刚写的本地状态）；若当前打开的会话被替换成消息更多的版本，面板直接升级它。合并结束后用一条「已从记忆体恢复 N 个会话」的提示告知用户，而不是静默改变列表。

测量。 客户端套件 `chat-mode-check.mjs` 用内存版 `localStorage` 与记录型 `fetch` 替身覆盖了这条路径的四个分支：在一个**空** `localStorage` 的全新 origin 上，记忆体里的会话被恢复出来并触发提示；一个比记忆体副本更新的本地副本**不会**被记忆体的旧副本覆盖；一个比记忆体副本更旧的本地副本**会**被升级；一份语法损坏的 JSON 载荷被跳过，且不影响其余会话

的列表渲染。四个分支合起来界定了这条修复的语义：它是**最终一致**的，不是强一致的——本地写入与记忆体写入不是同一个事务，因此两端在同一时刻可能看到不同结果，收敛发生在下一次启动合并时。

仍未脱离 origin 的部分。 需要强调两点边界。其一，本地 `localStorage` 仍然是**第一写入路径**（首屏同步渲染依赖它），因此清理站点数据并不会立刻清空屏幕，但它清空的是本地副本；恢复能力完全来自记忆体。其二，会话的**活跃状态**（当前打开的会话、输入草稿、未发送附件）仍然只存在于内存与本地缓存里，记忆体保存的是已完成轮次的快照，因此恢复的是历史，不是进行中的状态。这两点都是设计取舍，不是遗漏。

2.4 L4：历史图片从记忆体取回原始字节

问题。 v1 为节省 `localStorage` 配额，历史里的图片只保留一张 160 px 缩略图；重发一个旧轮次时，缩略图不足以作为图像输入，面板于是把该图片替换成一句「图片不可用」的说明。这个行为在 v1 里是**有意设计的**——它明确告知而不是静默丢图——但它使「继续一段含图的旧对话」在功能上不可行：用户必须重新上传图片，而重新上传的图片与历史里的那条消息不再有任何联系。

修复。 修复的关键是把图片的**字节**与图片在消息里的**位置**解耦。上传图片时，面板已经把字节存入记忆体以换取一个条目 id（`remember({ name, mime, bytes })` 返回 `storedId`）；v2 把这个 id 记在附件的 `memoryId` 字段上，并让可还原载荷一并保存它（见 2.3 的载荷形状）。发送前，`hydrate(history)` 遍历这条历史，对每一条**用户**消息里 `kind === 'image'` 且缺少 `data` 的附件调用 `restoreImage`：若附件自带 `data` 就直接返回；若有 `memoryId` 就 `GET /api/superman/file?id=<memoryId>`，响应成功且字节非空时把字节转成 base64 填回 `data`；若无 `memoryId` 或请求失败，返回原附件不变，发送路径随后回退到「图片不可用」的说明，且**该轮照常发送**，不会因为一张找不回的图而卡住会话。取回的字节按原来的媒体类型发给模型，不重新编码，因此模型看到的是用户当初上传的那张图，而不是缩略图。

测量。 `chat-mode-check.mjs` 新增的检查覆盖两条分支。成功分支：一个带 `memoryId` 且没有 `data` 的图片在发送前触发对文件路由的请求，**出站请求体里带有** base64 **字节**，并且该轮不出现「不可用」提示——这条检查同时验证了「真的取回了字节」与「真的用上了」，而不只是「发了一个请求」。失败分支：文件路由返回 404 时，回退提示出现，且该轮仍然发送成功。两条分支合起来把「不静默丢图」从 v1 的承诺变成了可执行的断言。

边界。 有四点需要说明。其一，`hydrate` 只处理用户消息里的图片附件，助手回合不携带附件。其二，`memoryId` 只对 v2 之后的上传存在：一份在 v1 时期写入、之后被恢复出来的历史里没有这个字段，因此这些旧图片仍然走「不可用」分支——这是向后兼容的代价，不是回归。其三，原始字节能否取回取决于记忆体是否仍然持有它：条目可能已经被淘汰，这正是 404 分支存在的原因；把 L4 的修复与 L5 的压缩策略放在一起看，一个被淘汰的图片会退回到显式的「不可用」提示，而不是一个损坏的图像块。其四，检查证明的是请求与请求体的形状，不是真实浏览器里的像素渲染——后者属于 L7 与讨论一节的效度威胁。

2.5 L5：记忆体的容量上限、淘汰、对账与手动清理

问题。 v1 的记忆体只有一个条目数上限，没有容量策略：文件与索引只增不减，唯一近似「去重」的机制是「相同 sha256 且同名的新上传刷新收录时间而不新建条目」。这意味着内容相同但文件名不同的材料会被存两次，而索引会随使用持续增长。对一份长期记忆来说，「只增不减」不只是磁盘问题，它还会稀释回忆：条目越多，同一查询命中的条目越多，摘要里塞进不相关材料的概率越高。

修复。 v2 给记忆体加了一整套压缩策略，由四部分组成。

1. **双上限。** `maxItems`（默认 2 000）与 `maxBytes`（默认 512 MiB）同时生效；单文件上限仍为 64 MiB。 2. **最旧优先淘汰。** 每次入库后执行一次 `compact(protectId)`：只要条目数超过 `maxItems` 或字节总数超过 `maxBytes`，就按 `addedAt` 升序取最旧的一条，把它从索引中移除、从磁盘上删掉，然后重新判断，直到两个上限都满足。刚刚入库的那一条被 `protectId` 保护，因此一次超大上传仍然能存进去，代价是它可能把更早的材料挤出去。淘汰结果随入库响应返回（`evicted`），并累加到统计里。 3. **启动对账。** 存储打开索引后执行一次 `reconcile()`：索引里有、`files/` 下没有这一 id 的条目被丢弃；`files/` 下有、没有任何条目引用的文件被删除；被条目引用的文件永不删除；一旦发生上述任一情况，索引重写一次。 4. **手动清理与计数。** 新增 `{action:'prune'}`，按同一个策略立即执行一次并返回 `{removed, freedBytes, items, bytes}`；`stats` 增加 `evicted`（本进程淘汰过的条目数）与 `orphanFiles`（对账删掉的孤儿文件数）。

测量。 Host 检查套件用五个独立的临时记忆目录覆盖这套策略，每条断言都指向一个可观察的事实，而不是策略的内部状态：在 `maxItems = 3` 时存入第 4 条，入库响应里的 `evicted` 恰好含 1 条且是最旧的那一条，它的文件从磁盘上消失、也从索引里消失，`stats.evicted` 变成 1；在 `maxBytes = 300` 时连续入库，每一次入库后的字节总数都不超过 300，最旧的两条被淘汰且它们的文件被删除，`files/` 目录里剩下的文件数与索引引用数一致；用手工构造的记忆目录（一条索引与文件都在、一条索引在文件不在、一个文件无索引引用）启动，对账后索引只剩有效的那一条、孤儿文件被删除、被引用的文件保留，且索引只重写一次、`stats.orphanFiles` 至少为 1；`prune` 在 `maxItems = 2` 的四条记忆上报告 `removed = 2` 与正确的 `freedBytes`，删除被淘汰的文件并保留其余。

写入的原子性同时被验证：并发 8 个入库请求全部成功，`index.json` 是合法且完整的 JSON（8 条），目录里没有残留的 `index.json.tmp`。这条检查针对的正是 v1 未加容量策略时同样存在、但更容易被并发放大的风险：索引是「读—改—写」的，如果没有串行化与原子替换，两个并发上传就会让其中一个消失，或者留下半个 JSON。原子替换与幂等写入在这里不是实现风格，而是把「崩溃与并发」变成可推理性质的手段 [38]。

边界。 三点。其一，淘汰是**按时间**的，不是按价值或使用频率的：一条三天前收录、之后被反复回忆的关键材料与一条三天前收录、从未被用过的临时文件，在策略眼中同样是「旧」。其二，去重规则没有改变，仍是「内容哈希 + 文件名」，因此同内容不同名的材料仍会被存两次——这一条在 v1 的局限里被点名，v2 没有关闭它，只是让它不再导致无界增长。其三，`maxBytes` 统计的是索引里记录的条目字节数，不是目录实际占用；两者在对账之后一致，在上传与淘汰之间可能短暂不一致。

2.6 L6：面板上传与 Harness 捕获的 PDF 在入库时即带文本

问题。 v1 的 PDF 抽取器只存在于浏览器半。Chat 上传的路径是完整的——面板在浏览器里抽出文本，把字节与文本一起提交给记忆体——但另外两条入库路径不是：面板的「上传」按钮只提交字节，Harness 侧的 `session/event` 监听器在捕获附件时也只提交字节。结果是同一个 PDF 从不同入口进来，记忆里的质量不同：从 Chat 进来的可以被正文检索到，从面板或 Harness 进来的只能靠文件名被检索到。这不是一个显眼的缺陷，它不会报错，只会让回忆悄悄变差。

修复。 我们把浏览器半的抽取器移植成 Node 模块 `superman/pdf.js`（799 行），并让 Host 半在**入库时**调用它。存储阶段取正文的规则是三级：若调用方已经给了文本（Chat 上传就是这种情况，并且它会截断到 4 000 字符预览），直接用；若分类结果是 PDF，就用 `extractPdfText(bytes, { maxPages: 200, maxTextChars: 400 000, maxChars: 400 000 })` 抽取，最多 200 页、最多 400 000 字符；若分类结果是程序或「其他」，把字节的前 32 KiB 按编码识别解码成文本预览；图片与其余情况不取正文。抽取整体包在异常处理里：**解析失败不影响入库**，条目照常写入，只是没有正文预览——「记得住文件」与「记得住内容」是两件事，前者不能因为后者失败而失败。

测量。 移植的正确性由一条**对等检查**保证，而不是由「看起来一样」保证：检查套件用同一批文件分别调用**浏览器半的抽取器**与 Host **半的** Node **模块**，要求两者的文本、页数与状态说明（`note`）完全一致——即逐字相同，而不是相似。用于对等的输入包括本机 `papers/` 目录下的真实 PDF 与一组合成 PDF（构造出的带文字层文档、对象流形式、CMap 损坏等结构），覆盖的正是 v1 时期暴露过缺陷的五类结构。此外，检查还验证了解析失败时字节仍然被存下：一条无法解析的输入不会让入库返回失败。

入库抽取把 v1 语料的全部测量带进了 v2 的语境：14 份文件、199 942 个字符、67 235 个汉字、138 页、替换字符与 C1 控制字符 0，其中 5 份抽到文本层、9 份被如实报告为 `no-text`（本机 OCR 的测量见表 3，抽取语料的逐文件数据见附录 C.1）。逐文件的字符数与页数见图 3，中位耗时与页数的关系见图 5。落在零点上的九个文件不是失败的重试对象，而是 L1 的入口：它们在文字层路径上得到诚实的结果，在 OCR 路径上可以换回正文。

表 3. v1 与 v2 的源码清单。

文件	v1 行数	v2 行数	增减
`chat-mode/client.js`	4,017	4,281	+264
`chat-mode/index.js`	255	255	+0
`superman/client.js`	1,320	1,320	+0
`superman/index.js`	666	908	+242
`superman/pdf.js`	—	799	新增
`superman-ocr/index.js`	—	383	新增
`tools/ocr-pdf.swift`	—	214	新增
`arxiv-search/client.js`	784	784	+0
`arxiv-search/index.js`	139	139	+0
total (module .js)	7,181	8,869	+1,688

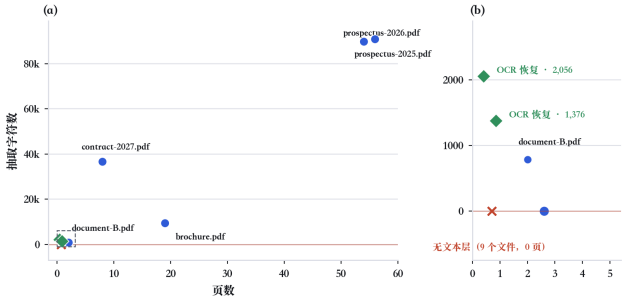


图 6. 语料逐文件的抽取字符数与页数：九份无文字层文件落在零点线上，OCR 可恢复的案例单独标出。

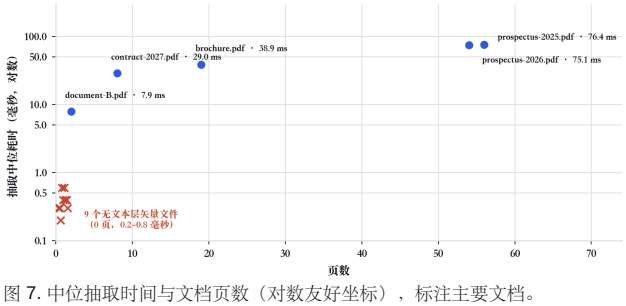


图 7. 中位抽取时间与文档页数（对数友好坐标），标注主要文档。

边界。 三点。其一，对等检查证明的是「两个实现一致」，不是「两个实现都正确」：如果两端共享同一个误读，检查不会发现。其二，移植使入库变慢：一个 400 000 字符的 PDF 现在在 Host 半被完整解析一次，而在 v1 里面板上传的 PDF 根本不在 Host 半解析。200 页与 400 000 字符两个上限界定了这一开销的上界，但它仍然是一项真实的成本。其三，抽取上限与浏览器半的上限不同（浏览器半按 400 000 字符截断，另有一层 4 000 字符的预览截断），因此「记忆里的正文」与「模型的上下文里能出现的正文」不是同一份文本，这一点在回忆摘要与提示词注入的语义里必须记住。

2.7 L7：浏览器级 DOM 验证——部分关闭

问题。 v1 的五套离线检查都用**桩 React 直接调用函数组件**，因此它们在 Node 里真实执行了组件体，却从未装配过真实 DOM：元素类型与属性可以被断言，事件冒泡、受控输入、焦点、`act()` 语义与渲染时序都不能。这是一个方法论缺口，不是产品缺陷，但它决定了两件事能否被验证——「面板真的能挂载」与「交互真的能驱动状态」。

修复（进行中）。 我们在工作区里建立了一套本地工具链来补这个缺口：`tools/dom/` 装有 jsdom [39]、React 与 ReactDOM（19.2 系列）的本地副本，`tools/dom-check.mjs`（884 行）用 `createRoot` 把面板真实挂载进 jsdom 文档，用真实 DOM 事件驱动它——`input` 与 `change` 设值、`keydown` 回车、`click` 点击、`change` 选文件——并等待真实的异步渲染结果。目前已经跑通的断言族包括：Chat 面板注册 main 槽并把自己的样式表写进文档 `head`；面板通过 `createRoot` 挂载出 `section.cm_root`；输入区渲染出真实的 `textarea`，输入的内容确实进入 `value`；发送按钮在草稿非空后启用；回车触发一次 `POST /api/chat-mode/stream`，请求体携带用户输入；发送后草稿被清空；「新建会话」按钮清空当前线程。

尚未收敛的部分。 完整套件目前**不能通过**：在 `act()` 与异步渲染的收敛过程中，发送之后的断言会超时——用户气泡与流式回答没有按预期出现在 DOM 里，脚本随后在「点击历史条目」处因找不到该元素而中止。这是 jsdom 与 React 的环境差异（`act` 的刷新时序、`console.timeStamp` 一类宿主能力、流式响应的调度），不是产品代码的失败：同一批面板文件在桩 React 套件里通过全部检查，在运行中的页面上也被人眼核对过。因此在 v2 发布时，`tools/dom-check.mjs` 与 `tools/dom/` **没有**纳入发布目录，它们留在工作区里作为下一步工作的起点。

状态的准确表述。 L7 是**部分关闭**：工具链存在、真实挂载与真实事件驱动已经实现、一部分断言已经稳定通过，但整套检查既不稳定也不完整，因此不能作为「DOM 层已验证」的证据。图 2 与表 4 把 L7 标为部分关闭，并在登记表里写明它当前停在哪一步；第 3 节把它列为最重要的效度威胁之一。

2.8 L8：语义回忆——未关闭

问题与结论。 v1 的最后一条局限是回忆**不是语义的**：它只捕捉表层相似度，「说法不同但意思相同」的材料不会被召回。v2 把打分升级为 BM25，但 BM25 仍是词法检索——它更好地衡量了「字面匹配得有多强」，没有改变「必须有字面匹配」。**这条局限没有关闭，而且不是代码问题。** 语义回忆需要两样东西之一：本机可用的嵌入模型，或一个可访问的向量服务（嵌入服务、向量数据库、或一个能充当重排器的模型端点）。这台机器上两者都不存在，套件又有一条贯穿始终的硬约束——不引入外部服务、不新开监听端口、不在后台偷偷联网。因此 v2 的选择是把 L8 如实保留为未关闭，并把接入点收敛成最小面。

为了接入而做的准备工作。 打分函数被隔离在单个 `recall(q)` 方法里，它读存储、切词、计算 BM25、叠加新鲜度项、裁剪到 `limit` 并渲染摘要。调用方只有两处：Chat 面板在发送前用本轮问题（截断到 600 字符）取一次回忆摘要，Host 半用它计算注入系统提示词的分段。两处都只依赖返回结构（`{matches[], digest}`），不依赖打分的内部形式，因此替换打分的改动范围就是 `recall` 一个函数体。这是有意的防腐层设计，也是把「未完成的局限」与「无法完成的工作」区分开的做法：我们无法在本机提供嵌入服务，但可以保证接入它不需要改动调用方。

词法检索在中文上的具体形态。 为了让这一缺口可检验而不是抽象，我们把它写清楚：查询先被归一化，拉丁词按非字母数字下划线加号点号切分并保留长度 2–40 的词；中文（含假名与谚文）逐字切分并加入相邻二元组；去重后作为词元集。因此查询「长期记忆」的词元是 ■/■■/■/■■/■/■■/■，一份写作「持久记忆」的文档的词元是 ■/■■/■/■■/■/■■/■，两者在 ■、■■、■ 上重合，可以命中——这看起来像语义匹配，实质上仍是字形重合的巧合：换成「记忆存储」与「长期保存」，重合会显著减少甚至消失。同日文与韩文同理。这一点是 L8 未关闭的直接后果，也是我们把 BM25 的改动描述为「更好的词法检索」而不是「语义检索」的原因。

如果接入，它会需要什么。 一条完整的嵌入路径至少需要三个组件：一个把文本编码为向量的模型（本地运行或按请求调用）、一个存放与检索向量的索引（用精确搜索在小规模记忆上已足够，规模上去以后需要近似最近邻），以及一个决定何时使用语义路径的开关（因为语义检索并非在每种查询上都优于词法检索，二者互补）。若只想在现有 BM25 之上做轻量改动，则可以用 BM25 取候选、再用一个模型端点对候选重排——这也需要同一个模型端点。三种方案都依赖本机现状不允许的外部依赖，因此它们都只能停留在设计层面。我们没有为了填补这一条而引入一个未经测量的近似方案；把一条未完成的局限留在明面上，比用一个未经验证的功能掩盖它更符合本报告的立场。

2.9 验证与代码的增长

八条局限的修复同时改变了被验证的东西。离线检查由 v1 的 284 项增长到 v2 的 449 项，套件由 5 套变为 6 套；代码由 7 187 行 JavaScript 增长到 8 869 行（含新增的抽取器与 OCR bundle）。v1 与 v2 的源码规模见表 1，两版的整体拓扑对照见图 1，各套件的检查项数见图 6，各模块的代码增长见图 8。

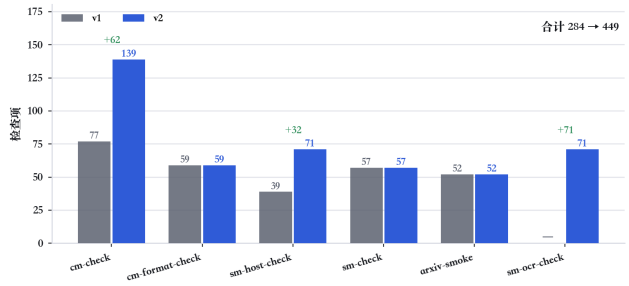


图 3. 各套件在 v1 与 v2 的检查项数分组柱图，标注总数。

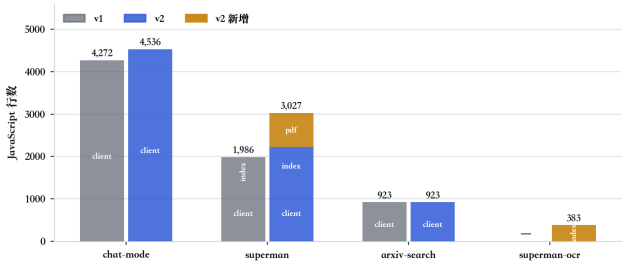


图 4. 各模块代码增长（v1 与 v2 堆叠），新文件高亮。

新增的检查集中在三处：记忆体 Host 半由 39 项增至 71 项（BM25、压缩、对账、prune、两端抽取器对等），Chat 管线由 77 项增至 139 项（可还原载荷与会话合并、图片字节取回、OCR 降级），以及一套全新的 OCR bundle 检查 71 项（路由与信封、参数校验、编译一次与缓存、超时、临时文件清理）。渲染与解码、记忆体面板、arXiv 三套没有增删检查，因为它们覆盖的能力在 v2 里没有改变——这一点本身是一个信号：修复集中在少数几个子系统，而不是一次全面重写。

图谱的两半也在同一时间被重新确认。运行中的页面上，三个侧边栏入口（`conversation` 在 −20、`chat` 在 −10、`superman` 在 −5）与随包的 `plugins`（0）并列生效；提示卡浮层以 `shell.overlay[superman.toast]` 注册，与随包的九个占用者并存；记忆体的动态提示词分段出现在**这套扩展自身所在的 Agent 会话的系统提示词里**，列出实时的记忆目录路径与计数（`PDF 2 · ■■ 0 · ■■ 0 · ■■ 0 · ■■/■■■■ 11`，随会话推进而增长）——也就是说，Harness 侧的回忆路径由本次会话本身在持续验证。v1 部署升级到 v2 的生效方式不同：浏览器半的修改由 500 ms 的 mtime/size 轮询热重载，立即生效；Host 半的修改（BM25、压缩、入库抽取）需要重启 Harness 进程。逐套件清单与每套驱动的内容见表 6，逐项差异与生效位置见表 5。

表 4. v1 与 v2 的离线检查套件清单，以及每套驱动的内容。

套件	用途	v1	v2	行数
`chat-mode-check.mjs`	对话管线：路由、SSE、历史、附件、记忆体联动	77	139	823
`chat-mode-format-check.mjs`	渲染与解码：LaTeX→MathML、Markdown、编码识别	59	59	263
`superman-host-check.mjs`	记忆体 Host 半：抽取、BM25 回忆、上限、对账	39	71	501
`superman-check.mjs`	记忆体面板：入口、文件夹、预览、上传、提示	57	57	348
`arxiv-smoke.mjs`	arXiv 包，含收藏路径	52	52	450
`superman-ocr-check.mjs`	OCR 包：路由、信封、首次编译、真实引擎	—	71	407
total		284	449	2,792

表 1. v1 与 v2 的差异：改了什么，以及在哪里生效（客户端热重载还是 Host 重启）。

#	方面	v1 行为	v2 变化	生效方式
L1	扫描件与 OCR	PDF extraction reads the text layer only; scans yield nothing	on-device OCR through macOS Vision, wired into the Chat upload path as an automatic fallback	client HMR + OCR bundle
L2	回忆打分	recall was a flat additive lexical score that favoured long documents	BM25 (k1 = 1.2, b = 0.75) with IDF over name/body/record fields plus the recency term	host, restart
L3	历史存储	conversation history lived in the page origin's localStorage	every finished turn also writes a restorable JSON payload (chatstate:<id>, ≤ 7 900 characters) that merges on start-up	client HMR
L4	图片保真	images in history kept only a 160 px thumbnail and resent as unavailable	an image remembers the memory item id of its bytes; a resend fetches the original from the file route	client HMR
L5	压缩策略	the memory grew without a compaction policy	maxItems 2 000 and maxBytes 512 MiB caps, oldest-first eviction, start-up reconciliation, prune, eviction counters	host, restart
L6	入库抽取	panel uploads and Harness captures stored PDF bytes without text	superman/pdf.js, a Node port of the browser extractor, parses on store (200-page / 400 000-character caps)	host, restart
L7	DOM 验证	no automated browser-level (DOM) end-to-end test	a workspace-local jsdom + React toolchain and tools/dom-check.mjs mount the panels with createRoot; environment differences remain	toolchain, not shipped
L8	语义回忆	recall is not semantic	no embedding service exists on this machine; retrieval stays lexical (now BM25), the seam is recall(q)	—

165 项新增检查覆盖了什么，没有覆盖什么。 增长的绝对数是 165 项（284 → 449），但把它当作覆盖面的度量会误导，因为不同检查的证明力差别很大。新增检查可以分成三类。

第一类：对不变式的断言（最有价值）。 压缩策略的检查属于这一类：它们断言「索引引用的每个文件都存在，且 `files/` 下的每个文件都被索引引用」这一不变式在正常路径上始终成立（淘汰同时删除条目与文件、字节上限在每次入库后成立），并且在被破坏后能被修复（对账）。这类检查的价值在于结论可以外推到检查没有覆盖的输入：只要不变式成立，多存一条或少存一条不会改变结论。

第二类：对可观察契约的断言。 可还原载荷与图片取回属于这一类：它们断言一个具体的请求被发出、请求体里带着具体的字段、以及某句提示出现或不出现在出站消息里。它们能证明链路接通与字段正确，不能证明视觉结果或模型行为——「请求体带 base64」与「模型看到的是那张图」之间仍然隔着一次真实调用。

一个具体例子可以说明这三类的差别。假设压缩策略的实现被改坏，使淘汰时只删索引项、不删文件：第一类检查会失败（它断言文件也被删除），错误被定位到「不变式被破坏」；第二类检查不会失败，因为可还原载荷与图片取回都不关心孤儿文件；第三类检查也不会失败。反过来，假设图片取回的请求体字段名被改错：第二类检查会失败，而第一、三类检查不会。检查的分类因此不是文档分类，而是**故障定位的坐标系**：它告诉我们某一次修改最可能在哪一类检查上暴露。

第三类：对实现对等的断言。 两端抽取器对等属于这一类，它的证明力与前提一样强：给定「浏览器半的实现是正确的」（这一前提由 v1 的语料测量支持），对等检查把这一性质传递给 Host 半。但它有两个盲点：两端共享同一个误读时检查不会发现；语料缺失时检查无法执行（本工作区当前即为此情形，见 3.2 的威胁三）。

另一类必须点明的是**未增删**的检查：渲染与解码 59 项、记忆体面板 57 项、arXiv 52 项，合计 168 项在 v2 中没有变化。它们的存在使「449 项全绿」这句话包含了对未改动能力的回归保护，因此把总数与「新能力的验证强度」直接挂钩是不准确的。

图 9 用两份文档、一个稀有查询词的算术示例，把 L2 修复前后的排序差别固定下来。

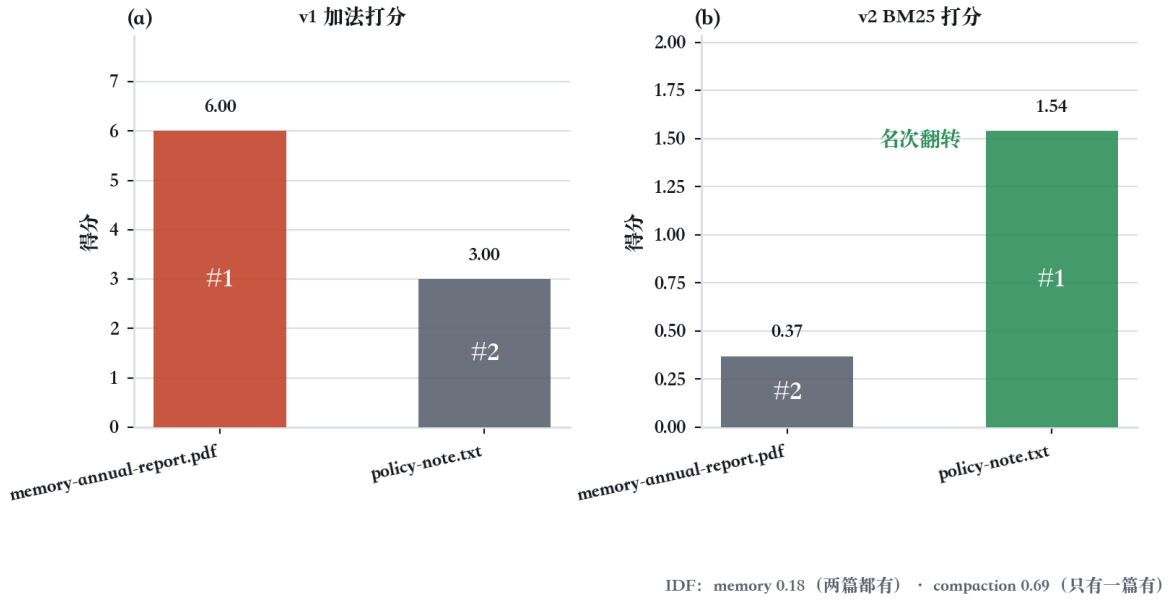


图 8. BM25 计算示例：两份文档、一个稀有查询词，加法打分与 BM25 给出相反的排序。

在线观察补了什么。 离线检查不能证明两件事：扩展在真实页面上的挂载，以及回忆在真实提示词里的出现。我们用 Inspection 查询补了这两点：三个侧边栏入口 active、`main` 的 `chat` 与 `superman` 面板 active、`superman.toast` 存在于 `shell.overlay`，以及运行中 Agent 自身的系统提示词里出现记忆体摘要与实时计数。这类观察是单次快照、样本量为 1，因此它证明「链路在至少一次真实运行中通」，不证明「在每次运行中都通」。

3 讨论

3.1 关闭局限的实践教益

这次工作给出八条可迁移的经验。

第一，逐条登记局限，比笼统承认局限有用得多。 v1 的八条局限写在讨论一节的末尾，形态是六条项目符号；到了 v2，每一条都有了编号、修复方式、可观察判据、证据与状态（附录 D）。差别不在于措辞的严谨，而在于**编号让「关闭」变成可分配、可完成、可验收的动作**：一条局限一旦有编号，就可以被分配一个修复、一条检查、一个结论，而不是长期停在「已知问题」的清单里。反过来，「没有编号的局限」在本项目里也确实更容易被拖着——L7 与 L8 之所以没有关闭，一部分原因正是它们看起来不像可以被单独完成的任务。

第二，修复的方向往往是「补一个已经在别处存在的东西」。 六条关闭的局限中，三条的修复是把已有的能力接到缺失的路径上，而不是发明新能力：OCR 用的是系统自带引擎，BM25 用的是标准公式，入库抽取是把浏览器半的抽取器移植到 Host 半。这与「把能力抽象成可替换的接缝」是同一件事的两面：正因为 PDF 抽取是一个独立函数、回忆是一个 `recall(q)` 函数，把它们移植或替换才是低风险的改动。系统设计上的小决定——函数边界在哪里——在局限修复阶段决定了成本。

第三，必须接受「修复会引入新的代价」。 每条修复都带来了新的失败模式或新的成本：OCR 带来识别误差与首次编译延迟，BM25 带来整库遍历的常数开销与中文分词的偏差，可还原载荷带来「丢轮次还是丢 JSON」的取舍，图片取回带来 404 回退路径，压缩策略带来可能淘汰仍有价值材料的风险，入库抽取带来 Host 半的解析开销。把代价写进正文而不是留在提交记录里，是这次工作里最容易被忽略、也最影响可信度的一步。

第四，验证套件的证明边界必须与被验证的能力同步更新。 449 项检查是增长了，但增长的意义取决于它们覆盖了什么：BM25 的三文档检查覆盖了稀有词与长度偏差，但它不证明 BM25 在通用检索基准上的排序质量；OCR 的检查覆盖了路由、编译、超时与清理，但它不证明识别准确率；对等检查证明两个抽取器一致，不证明它们正确。把「449 项全绿」当作终点，会把一套精心设计的方法学消耗成一句口号。这也是我们坚持在正文里写明「这些检查证明不了什么」的原因。

第五，未完成的部分需要说明它为什么未完成。 L8 未关闭的直接原因是本机没有嵌入服务，但更根本的原因是套件的一条设计约束——不引入外部服务、不新开端口、不在后台联网。约束与局限在这里是同一件事的两面：正因为记忆体是本地优先的，它才便宜、确定、可离线验证；也正因为它是本地优先的，它才无法获得语义能力。把这一因果写清楚，比声称「未来会加上语义检索」更有价值。

第六，「已关闭」是一个带条件的判断，条件必须一起写出来。 六条已关闭的局限里，没有一条是无条件的。L1 的前提是「本机是 macOS 且装有 Vision」——换一个平台，这条修复就不存在，套件退回原来的「无可回忆文本」行为；L2 的前提是「记忆体里有足够多的文档让 IDF 有意义」，只有一份材料时 BM25 与加法打分没有区别；L3 的前提是「记忆体可用」，记忆体缺席时历史仍退回 `localStorage`；L5 的前提是「使用者接受按时间淘汰」，换一个需要保留全部历史的使用场景，2 000 条上限就是错的默认值。因此正确的表述不是「这六条已解决」，而是「在这台机器、这套配置、这些判据下，这六条已经被解决」。把条件与方法学放在一起，读者才能判断这些结论在多大程度上可以迁移。

第七，「关闭」不是终点，它有一个半衰期。 这一次的修复有相当一部分是**替换**而不是**新增**：BM25 替换了加法打分（原有的打分代码不再存在），入库抽取替换了「只存字节」的路径，可还原载荷替换了「散文式记录」这一唯一表示。被替换的实现不会自己保持正确——它只是不再被调用。随着宿主与依赖演进，这些修复会以两种方式失效：其一是外部环境改变（系统 OCR 引擎的接口或语言包变化、`swiftc` 的行为变化），其二是宿主内部契约改变（`ctx.connection.fetch.register` 的参数或 `session/event` 的事件形状变化）。局限登记表与检查套件因此不是一次性的交付物，而是需要随宿主版本重新运行的东西；本次修复的检查里，若干条（路由形状、事件名、附件引用结构）恰好就是在守护这些外部契约。

第八，真实使用是最后一道验证，而它只覆盖了部分路径。 这套扩展在开发期间被用于自己的开发：数份 PDF 经记忆体收录，一段 5.0 MB 的 arXiv 预印本通过「收藏到记忆体」按钮存入，记忆体的摘要出现在本会话自身的系统提示词里。这条证据的价值与它的局限同样明显：它证明了记忆体在真实任务分布下会被触发，也证明了它被触发的场合恰好是「开发者在开发一个记忆体」——任务类型高度自相似（大量关于本套件自身的对话与文件），因此它对「记忆体在陌生领域里的召回质量」几乎没有信息量。我们把它作为「链路可用」的证据保留，而不作为「记忆有用」的证据。

3.2 效度威胁

威胁一（最重要）：DOM 层仍未验证。 449 项检查中没有任何一项装配真实浏览器 DOM；L7 的工具链能挂载、能驱动事件，但整套还没有通过（2.7）。因此所有关于**视觉与交互**的结论——布局、滚动、字体回退命中、拖拽与粘贴、提示卡的实际位置、`act()` 之外的渲染时序——都**未经核实**。这是本报告最重要的边界，它不因为检查数从 284 涨到 449 而缓解。

威胁二：OCR 的验证样本不完整。 三个案例覆盖了「纯栅格化」「栅格化且文本层近乎为空」「内嵌 CMap 损坏」三种情形，但它们是 3 个页面级的观测，不是一套 OCR 基准：没有平行文本（ground truth）对照，因此无法给出字符错误率或字错误率的定量值，只能给出「文本层对 OCR」的字符数对照与定性误差形态。第三例（字符多但汉字少）说明字符数本身也可能误导。此外 OCR 依赖本机 macOS Vision，其版本与模型不在我们的控制范围内。

威胁三：测量本身的稳定性。 PDF 抽取的耗时测量在多次运行之间并不完全可复现：同一份文档在不同次测量中有时相差 20–30 ms，抽取文本的长度也可能因子集差异而变（例如同一份文件一次抽出 199 942 个字符、另一次 227 522 个）。因此本文把抽取**字符数**当作可比量、把**耗时**当作指示量，并且不把单次测量的毫秒差解释为版本之间的性能差异。

威胁四：语料规模与来源。 抽取语料为 14 份本机文件：两份中文招生简章、一份合同、一份成绩单、一份招贴与九份 Apple 系统生成的矢量 PDF。中文文档集中于两份招生简章，不能代表 PDF 生态的多样性；九份无文字层的文件来自同一来源，可能共享同一种结构。编码探测只有 7 个探针，验证的是「给出正确标签」，不是长文本上的稳定性。

威胁五：单机、单会话、自证。 全部数字来自一台机器、一个工作区；在线观察是部署快照；「记忆体摘要出现在运行中 Agent 的系统提示词里」这一最强证据，恰好来自开发这套扩展的会话本身，属于自证。它证明了链路通，不证明它在别的任务分布上同样有用。

威胁六：BM25 示例是算术，不是实验。 图 9 的两文档示例由作者构造，用于说明两个打分函数的差异方向；它不是检索基准，也不声称 BM25 在通用语料上优于加法打分。真正的证据是检查套件里的四条断言，它们覆盖方向性结论而不覆盖绝对质量。

3.3 仍未关闭的部分（如实说明）

L7：部分关闭。 工具链存在、真实挂载与真实事件驱动已经实现、部分断言稳定通过；整套检查既不稳定也不完整；`tools/dom-check.mjs` 与 `tools/dom/` 未纳入发布目录。因此「DOM 层已验证」这一结论**不成立**。

L8：未关闭。 本机没有可用的嵌入或向量服务，套件的约束不允许引入外部服务，因此回忆仍是词法检索（已升级为 BM25）。接入点被收敛在 `recall(q)` 一个函数里，使未来的接入不需要改动调用方；但接入本身没有发生，因此任何关于语义召回能力的说法都没有依据。

此外，v1 列出的六条局限中，有一条只被部分缓解：记忆体的去重仍是「内容哈希 + 文件名」，同内容不同名的材料仍会被存两次；v2 只是让它不再导致无界增长。我们在附录 D 的登记表里把它与 L1–L8 一并标出，避免读者把「压缩策略存在」误读为「去重问题已解决」。

3.4 本节结论

回到最初的问题：一个不依赖任何外部服务的本地记忆，在智能体框架里能做到哪一步？本次工作给出的答案是：**它可以在六条具体的能力缺口上达到可用，并且这些缺口可以逐条被测量**。扫描件从「零字符」变为可检索文本，代价是识别误差与首次编译延迟；回忆从「偏向长文档的加法打分」变为带长度归一化与 IDF 的词法排序，代价是整库遍历与中文分词的偏差；会话从「只在本地」变为「本地优先、记忆体兜底」，代价是最终一致而不是强一致；图片从「只留缩略图」变为「按 id 取回原图」，代价是一条 404 回退路径；记忆体从「只增不减」变为「有上限、可对账、可清理」，代价是按时间淘汰的粗糙；入库从「只有 Chat 带文本」变为「所有入口都带文本」，代价是 Host 半的解析开销。六条修复合起来把「验证规模」从 284 项推到 449 项，同时把两条仍然开放的缺口（DOM 层验证、语义回忆）留在了明面上。

这份清单的价值不在于它有多长，而在于每一项都可以被独立复核：判据写在正文里，断言写在检查里，代价写在边界里。对同类系统而言，可迁移的做法是同一个：**先把局限编号，再为每条编号规定一个可观察的判据，最后把判据写进随代码发布的检查**。本文的八条经验与两处未完成，是这一做法在真实项目上的一次执行记录。

4 方法

4.1 插件模型与运行环境

套件是若干 Cordis [2] bundle，每个 bundle 由 Host 半（Node 模块）与浏览器半（ES 模块）组成，通过 `apply(ctx, config)` 接入宿主上下文；安装通过 profile 的 bundle 列表完成，宿主源码零改动。我们使用的服务有 `ctx.slots`（界面槽位）、`ctx.connection.fetch`（已鉴权 Fetch 路由注册）、`ctx.llm.stream`（模型流式调用）、`ctx.attachments`（附件存取）、`ctx.systemPrompt`（系统提示词分段）与 `ctx.on('session/event')`（会话事件订阅）。界面全部通过 `ctx.slots.inject(key, cb)` 与 `ctx.slots.register(options, component)` 注入，只增不改：`sidebar.panellist` 新增 `conversation`（−20）、`chat`（−10）、`superman`（−5），`main` 新增 `chat` 与 `superman` 两个整页组件，`settings.section` 新增 `arxiv`，`shell.overlay` 新增 `superman.toast`。

传输方面，两个 Host 半都通过 `ctx.connection.fetch.register` 在已鉴权的 `/api` 通道上注册精确路径的 Fetch 路由，而不是新开监听端口：`POST /api/chat-mode/stream`（一次普通模型调用）、`POST /api/superman/api`（记忆体命令）、`GET /api/superman/file?id=…`（原文件字节）、`POST /api/superman-ocr/api`（OCR 命令）。这样做继承会话凭据与 Host/Origin 信任边界，不与桌面应用或 `dsh web` 争用端口，并随插件卸载自动撤销；代价是不能声明通配路径、`requestBody` 必须为 `buffered`，且 SSE 的读端必须由页面自行实现，帧分隔符（`\n\n`）因此成为契约的一部分 [4]。Chat 的 Host 半把模型流序列化为 `text/event-stream`，帧类型有 `text`、`reasoning`、`usage`、`finish`、`error`；路由把 `request.signal` 同时传给上游调用与帧写入函数，因此中止请求会真正终止上游计费流。

arXiv 检索的取数不由浏览器直连 `export.arxiv.org`——该端点不返回跨域头——而由 bundle 的 Host 半在同一已鉴权通道上代理并原样回传 Atom feed，查询按 arXiv API 用户手册 [40] 构造；相邻两次上游请求默认排队 3 000 ms。PDF 下载则相反：`arxiv.org` 的 PDF 响应带 `access-control-allow-origin: *`（以 HEAD 请求实测），因此页面可以自行取字节，**不需要**为 PDF 再加一条 Host 代理。

4.2 离线验证方法学

验证的目标是「在 Node 里驱动**随包发布的真实文件**」，断言真实代码路径与真实数据，而不是替身的返回值。浏览器半的装载方式是：桩 `window.__ModuleLoader__` 捕获 `load()` 调用，工厂函数以桩 `require('react')` 调入；桩 React 只实现 `createElement`、`memo`、`Fragment`、`useState`、`useRef`、`useEffect`，并**直接调用函数组件**（而不是模拟渲染），因此组件体真实执行、返回的节点树可在 Node 中断言；`document`、`navigator`、计时器（`unref` 化，避免轮询器挂住进程）、内存 `localStorage`（可设配额）与被替换的 `fetch`（按命令返回罐头响应）都是替身。Host 半运行在一个伪 Cordis 上下文（`inject`、`effect`、`on`、`get`、`connection.fetch.register`）里，记忆目录由 `mkdtemp` 创建、测试结束删除，系统提示词分段用一个记录型替身捕获。

真实的部分：插件源码、契约校验、状态机、PDF 抽取、公式生成、编码探测、记忆索引与文件、SSE 重组、URL 与请求体、DOM 结构（元素类型、属性与文本）。**证明不了的部分**：像素级外观、真实浏览器的字体与布局、真实 provider 的模型输出、真实 Harness 的鉴权链路（`/api` 的信任栅栏在本机由宿主服务保证），以及「提示卡确实出现在正确位置」这类视觉结论。检查在 CI 意义上的可复现性受限：记忆体 Host 套件中的真实 PDF 对等检查需要一个含 PDF 的 `papers/` 目录（见附录 E），缺少语料时该项无法通过，因此复现时应先确认语料就位。

这套方法学有四个刻意的选择，每个都对应一类它无法发现的问题。**第一，函数组件被直接调用而不是渲染**：这样做的好处是组件体真实执行、返回的节点树可以在 Node 里断言，代价是 React 的调度、批处理与副作用清理都不参与，因此「组件在真实渲染下是否重复请求」这类问题检查不到。**第二，`fetch` 被替换为按命令返回罐头响应的替身**：这使请求的形状（URL、方法、请求体字段）成为可断言的对象，同时也意味着响应内容由测试决定，因此「真实服务端返回的字段名与客

户端预期不一致」这类问题检查不到。**第三，计时器被 `unref` 处理**：这避免了轮询器挂住测试进程，但也意味着任何依赖真实时间流逝的行为（去抖窗口、4 秒轮询、7 秒提示卡消失）都只能断言「注册了定时器」，不能断言「在真实时间轴上按时发生」。**第四，Host 半的鉴权链路不在测试范围内**：`/api` 的信任栅栏由宿主服务保证，检查只验证路由注册到了这条通道上，因此「未鉴权的请求会被拒绝」这一性质不由本套检查证明。把这四点写在方法里而不是留给读者推断，是这套验证方法学的一部分：它把「证明边界」变成一份可以随代码一起维护的清单。

4.3 OCR 流水线

这条流水线要处理的文档有一类共同特征：它们往往是为长期保存或打印而生成的，例如归档格式的 PDF/A [41] 文件、排版工具导出的轮廓化稿件、以及由扫描仪直接产生的图像页。这类文档的文字层要么不存在，要么只承载一份并非正文的文本，因此文字层抽取与 OCR 并非互相补充的两种「读法」，而是互为前提的两个阶段。`tools/ocr-pdf.swift`（约 210 行）用 PDFKit [32] 渲染页面、用 Vision 的 `VNRecognizeTextRequest` [33] 识别文本。渲染比例默认 2 设备像素／PDF 点，可用 `--scale` 调整（钳制在 0.5–6）；最长边超过 4 000 像素时按比例缩小，以免单张位图耗尽内存；位图先填白底再绘制页面，避免透明底上的字形识别不良。识别请求的配置为 `recognitionLevel = .accurate`、`recognitionLanguages = ["zh-Hans", "en-US"]`、`usesLanguageCorrection = true`；结果按行的垂直中心降序、同行按左边界升序排序（Vision 的包围盒原点在左下，故 y 越大越靠上），以恢复阅读顺序。每页识别完成后立即写出 `## page N` 标记与该页文本行，因此超时被杀的进程仍留下已完成的页面；无法渲染的页面写到 stderr 并跳过，全部页面都无法渲染时以非零状态退出。

`superman-ocr` bundle 把它包装成一条路由与两个动作（`health`、`ocr`）。请求校验的顺序是：`base64` 必须存在且非空 → 长度不超过 `ceil(32 MiB / 3) × 4 + 8` 字符 → 解码后非空且不超过 32 MiB → 前 5 字节必须是 `%PDF-` → 请求未被中止。页数由 `maxPages` 钳制在 1–200（默认 40）。二进制按需编译：优先复用工作目录中已存在、且 mtime 晚于源文件的产物；否则用 `swiftc -O <■■■> -o <■■■■> -framework Vision -framework PDFKit -framework AppKit` 编译，并把 `CLANG_MODULE_CACHE_PATH` 与 `SWIFT_MODULECACHE_PATH` 指到工作目录内，使构建自包含；编译超时 300 000 ms，成功后 `rename` 安装并 `chmod 0755`。**每个进程内至多编译一次**：并发调用共享同一个进行中的 Promise。单次识别的时间预算为 600 000 ms，输入写入工作目录下的临时文件并在 `finally` 中删除，返回文本截断到 400 000 字符，页数由输出中 `## page N` 标记的个数统计。错误码到 HTTP 状态的映射是固定的：`bad-request` → 400、`too-large` → 413、`aborted` → 499、`ocr-timeout` → 504、`ocr-failed` → 502、其余 500；面板把错误码措辞成用户语言，而错误码本身保持稳定。

OCR 的三个参数取值有各自的依据：识别层级取 `.accurate` 而非 `.fast`，因为在扫描件上前者对中文的识别质量明显更好，而扫描件的使用频率远低于普通文本，多花的算力换来的是「本来就很难读的文档读得更准」；语言集合取简体中文与美式英文，因为本工作语料与使用者的文档以这两种语言为主，增加语言会抬高单页耗时并可能引入跨语言混淆；语言纠正打开，因为识别对象是成句的正文而非代号或代码。渲染比例默认 2 而不是更高，原因是它在「小字号可辨认」与「单页位图内存」之间取平衡：一份 A4 页面在 2 倍比例下约为 1190 × 1684 像素，识别质量已经稳定；`--scale` 允许用户在极端文档上自行调整，但上限 6 是为了防止一张位图耗尽内存。

Chat 半的调用策略是 `readByOcr`：只有在 PDF 抽取结果为空时才发起请求；请求带 600 000 ms 的 AbortController 超时；响应必须是 `ok: true` 且 `value.text` 为非空字符串，否则返回 `null`；返回 `null` 时面板保留原有的「无文本层」提示，不报错。识别成功时正文前插入一句标注「以下内容由本机 OCR 识别，可能有错字」，并标记 `note: 'ocr'` 与 `recognised = true`，使面板不再触发「文字过稀」的提示，同时把文本一并存入记忆体。识别质量与误差的度量方式在 2.1 讨论：序列解码的经典形式见 Rabiner [36]，字词层面的误差用编辑距离衡量 [37]；本文不给出字符错误率，因为没有平行文本。

4.4 BM25 的形式化

记 D 为整库文档数，$\mathrm{df}(t)$ 为含词元 t 的文档数，$f(t,d)$ 为词元 t 在文档 d 中的带权词频，$L(d)$ 为 d 的带权长度，$\bar{L}$ 为整库平均带权长度。IDF 采用 $\mathrm{IDF}(t) = \log\left(1 + \frac{|D| - \mathrm{df}(t) + 0.5}{\mathrm{df}(t) + 0.5}\right)$；单个词元的贡献为

$$w(t,d) = \mathrm{IDF}(t) \cdot \frac{f(t,d) \cdot (k_1 + 1)}{f(t,d) + k_1 \cdot \left(1 - b + b \cdot \frac{L(d)}{\bar{L}}\right)}$$

其中 $k_1 = 1.2$、$b = 0.75$。文档的 BM25 分数是全部查询词元贡献之和：$\mathrm{score}(d) = \sum_t w(t,d)$，最终排序分数是 $\mathrm{score}(d) + \mathrm{recency}(d)$，其中 $\mathrm{recency}(d) = \min\left(1.5, \max\left(0, 1 - \frac{\Delta t}{30\ ■}\right)\right)$，$\Delta t$ 是收录（或记录更新）至今的毫秒数。名称（或记录标题）以权重 3、正文以权重 1 进入 `f` 与 `L`：名称里的每一次命中因此等价于正文里的三次命中。文档频率按整库现算，因此整库只有 1 份时 IDF 恒为 $\log(1 + 0.5/1.5) = 0.2877$，词元之间无区别——这一点在单独存放一条记忆时是可接受的，也是 BM25 需要「有语料」才体现价值的原因。

词元化规则：文本先转小写；拉丁词按非 `[a-z0-9_+#.]` 的字符切分，保留长度 2–40 的词；中文、假名与谚文（U+3400–U+9FFF、U+3040–U+30FF、U+AC00–U+D7AF）逐字切出，并为每个字与它的后继字追加一个二元组；最后去重。回收时按分数降序取前 `limit` 条（默认 6，钳制在 1–20），命中片段取首个命中词元周围的 200 字符窗口（向前多取 60 字符并加省略号），空白压平。`recall('')` 或无法切出任何词元时返回空结果，摘要为空字符串。全部定义与实现一一对应，`internal` 导出暴露了 `tokenize`、`idf`、`documentVector`、`bm25Rank`、`recencyBonus` 供检查直接调用。

4.5 存储模式、原子性与压缩策略

记忆目录是 `dshHomePath('superman')`，默认 `~/.dsh/superman`，结构为 `index.json`（`{ version, items[], notes[] }`）与 `files/<id>`（id 为 sha256 摘要的前 16 位十六进制）。单一文件的写入是**原子替换**：把新内容写到 `<index>.tmp` 再 `rename` 覆盖，因此中途终止不会留下半个 JSON [38]。整个「读—改—写」循环被串行化：每次变更排在前一次变更之后，避免两个并发上传丢弃对方的条目。索引带版本号，不匹配或解析失败时从空索引重建，`files/` 下的原文件保留。

`Item` 记录 id、名称、分类、媒体类型、字节数、收录时间、来源（`chat/harness/superman/arxiv`）、来源标识、`detail`（人类可读大小或像素尺寸）、正文预览与其长度，以及可选的宽高与页数；`Note` 记录 id、`kind`（`task/conversation`）、标题、正文、来源、会话 id、创建与更新时间、标签。上限：预览每条 4 000 字符、记录正文 8 000 字符、单文件 64 MiB、条目 2 000 条、总量 512 MiB、记录数 500 条。

压缩策略由 `compact(protectId)` 实现：计算条目字节总数，当条目数超过 `maxItems` 或字节总数超过 `maxBytes` 时，按 `addedAt` 升序选出最旧且 id 不等于 `protectId` 的一条，把它从索引移除、从磁盘删除，累加淘汰计数，重复直到两个上限都满足；`protectId` 保证刚入库的条目不会被自己挤掉。启动对账由 `reconcile()` 实现：丢弃 id 为空或文件缺失的条目，删除 `files/` 下无条目引用的文件，重写索引一次，并记录孤儿计数。`prune` 动作按同一策略立即执行并把结果返回为 `{removed, freedBytes, items, bytes}`；`stats` 返回 `{total, bytes, notes, categories, evicted, orphanFiles}`。写入与变更的幂等性来自两点：条目按「sha256 + 名称」去重，重复入库只刷新收录时间；`prune` 与 `reconcile` 在无事可做时不写文件 [38]。

4.6 入库抽取与跨实现对等

`superman/pdf.js` 是浏览器半抽取器的 Node 移植：对象表扫描 → 流过滤器逆序应用（`FlateDecode` 走平台解压并容忍尾部填充、`ASCII85Decode` 与 `ASCIIHexDecode` 自实现）→ `/ToUnicode` CMap 还原 → 内容流算符重放（`Tf`、`Tj`、`'`、`"`、`TJ`、`Td`、`TD`、`T*`、`Tm`、`BT`、`ET`）→ 按页装配（沿 `/Parent` 上溯继承的 `/Resources`，解析 `/ObjStm` 对象流）。OCR 复核的数学表述无需额外机制：识别输出与文字层同为字符串，因此两者可以在同一接口下比较与存储。Host 半调用它的上限是 200 页与 400 000 字符，抽取异常被捕获，失败不影响入库。对等检查的形式是：对同一份字节，分别调用浏览器半与 Host 半的抽取入口，比较文本、页数与 `note` 三者是否完全相等；只有三者全等才算通过，任何一处不同都会把该文件的摘要写进失败详情。

4.7 DOM 工具链与复现

DOM 验证的工具链是工作区本地的 `tools/dom/`：`jsdom` [39] 提供文档与事件，`react` 与 `react-dom` [3] 提供真实渲染，`tools/dom-check.mjs` 用 `createRoot` 把面板挂载进 `document`，用 `fire`、`type`、`press`、`click`、`attach` 等辅助函数派发真实事件，并用 `act()` 与 `settle()` 等待 React 的异步刷新。该套件当前不能通过（2.7），因此不随发布目录分发。复现命令、仓库布局（`v1/` 与 `v2/` 并列）与各套件的运行方式见附录 E。为避免读者误以为「检查数增长」等价于「覆盖面增长」，附录 B 逐套件列出每套检查驱动的对象。

本次工作的代码与文档以 Git [42] 管理，套件源码以 MIT 许可证 [43] 发布；仓库与发布流程使用 GitHub 的仓库与内容 API [44]。文中的数学公式以纯 Unicode／拉丁文书写，排版时由 STIX 科学字体渲染 [8]；系统的排版语义参照 The TeXbook [45]。验证与数据结构上的一个细节来自循环不变式式的推理传统 [46]：`compact` 的不变式是「索引引用的每个文件都存在，且 `files/` 下的每个文件都被索引引用」，`reconcile` 与 `prune` 都保持该不变式，检查因此可以断言它的两个方向。

数据可用性

全部测量数据来自本工作区的脚本与源码统计，随稿以 JSON 与图表的中间数据形式提供：语料逐文件的字节数、抽取字符数、汉字数、页数、噪声字符数与中位耗时；编码探测的 7 个探针；16 条 LaTeX→MathML 表达式电池的标记长度与节点计数；三个来源模块的文件数、行数与字节数；六套检查的检查项数与脚本行数。语料 PDF 本身不在随稿数据中，因为其中包含第三方文件；论文与代码中使用化名，真实文件名不公开。OCR 的三例测量（文本层字符数与 OCR 字符数／汉字数）记录在随稿数据与附录 C 中。全部数字可用附录 E 的命令在本机复现。

代码可用性

扩展套件以两个版本并列发布：`v1/` 是 v1 发布时的代码，`v2/` 是本文描述的当前代码，两者在同一仓库中，便于逐条对照局限的修复 [42]。源码以 MIT 许可证发布 [43]，仓库与内容的发布流程使用 GitHub 的仓库与内容 API [44]。复现全部检查与图表生成所需的命令见附录 E。

致谢

感谢 DeepSeek Harness 与 Cordis 的作者提供可扩展的宿主与插件框架，感谢 Apple 提供 PDFKit 与 Vision 框架，感谢 jsdom、React 与 STIX 项目提供 DOM 验证工具链与科学字体。本文全部实测数字来自本工作区，未使用外部基准；文中引用的文献列表用于定位方法学与设计依据，不用于与他方系统作性能比较。

作者贡献

Anstar, Z. 设计并实现 v2 的全部修复（OCR 流水线、BM25 回忆、可还原会话、图片取回、压缩与对账、入库抽取），编写并运行验证套件，进行测量与图表生成，撰写论文。Zeng, P. 设计并实现 v1 套件，提供 v1 的全部实测数据与代码基线，审阅 v2 的接口兼容性与局限登记。DeepSeek Harness Contributors 提供宿主与插件框架的使用反馈与缺陷报告。全体作者审阅并同意最终稿件。

利益冲突

作者声明无相互竞争的经济利益。

参考文献

1. DeepSeek Harness — https://github.com/deepseek-ai/deepseek-harness
2. Cordis plugin framework — https://github.com/cordiverse/cordis
3. React — https://react.dev/
4. WHATWG, HTML Living Standard, § Server-sent events.
5. W3C, MathML Core, W3C Recommendation.
6. KaTeX — https://katex.org/
7. MathJax — https://www.mathjax.org/
8. STIX Two fonts — https://github.com/stipub/stixfonts
9. The Unicode Consortium, The Unicode Standard, Version 15.
10. WHATWG, Encoding Standard (GBK, Big5, Shift_JIS, EUC-KR decoders).
11. K. Sparck Jones, A statistical interpretation of term specificity and its application in retrieval, J. Doc. 28(1), 1972.
12. S. Robertson and H. Zaragoza, The probabilistic relevance framework: BM25 and beyond, Found. Trends Inf. Retr. 3(4), 2009.
13. S. E. Robertson, S. Walker, S. Jones, M. M. Hancock and M. Beaulieu, Okapi at TREC-3, TREC 1994.
14. WHATWG, HTML Living Standard, § Web storage (localStorage).
15. ISO 32000-1:2008, Document management — Portable document format — Part 1: PDF 1.7.
16. Adobe Systems, PDF Reference, 6th edn (FlateDecode, ToUnicode CMaps), 2006.
17. N. F. Liu et al., Lost in the middle: how language models use long contexts, TACL 2024.
18. C. Packer et al., MemGPT: towards LLMs as operating systems, 2023, arXiv:2310.08560.
19. J. S. Park et al., Generative agents: interactive simulacra of human behavior, UIST 2023.
20. S. Yao et al., ReAct: synergizing reasoning and acting in language models, ICLR 2023.
21. T. Schick et al., Toolformer: language models can teach themselves to use tools, NeurIPS 2023.
22. G. Salton and C. Buckley, Term-weighting approaches in automatic text retrieval, Inf. Process. Manag. 24(5), 1988.
23. V. Karpukhin et al., Dense passage retrieval for open-domain question answering, EMNLP 2020.
24. N. Reimers and I. Gurevych, Sentence-BERT: sentence embeddings using Siamese BERT-networks, EMNLP 2019.
25. O. Khattab and M. Zaharia, ColBERT: efficient and effective passage search via contextualized late interaction over BERT, SIGIR 2020.
26. P. Lewis et al., Retrieval-augmented generation for knowledge-intensive NLP tasks, NeurIPS 2020.
27. J. Johnson, M. Douze and H. Jégou, Billion-scale similarity search with GPUs, IEEE Trans. Big Data, 2019.
28. Y. A. Malkov and D. A. Yashunin, Efficient and robust approximate nearest neighbour search using hierarchical navigable small world graphs, IEEE TPAMI, 2018.
29. A. Vaswani et al., Attention is all you need, NeurIPS 2017.
30. J. Devlin et al., BERT: pre-training of deep bidirectional transformers for language understanding, NAACL 2019.
31. T. Brown et al., Language models are few-shot learners, NeurIPS 2020.
32. Apple Inc., PDFKit framework, developer documentation.
33. Apple Inc., Vision framework — VNRecognizeTextRequest, developer documentation.
34. OpenAI et al., GPT-4 Technical Report, 2023, arXiv:2303.08774 (used only as a corpus sample).
35. R. Smith, An overview of the Tesseract OCR engine, ICDAR 2007.
36. L. R. Rabiner, A tutorial on hidden Markov models and selected applications in speech recognition, Proc. IEEE 77(2), 1989 (the classic OCR/sequence-decoding reference).
37. V. I. Levenshtein, Binary codes capable of correcting deletions, insertions and reversals, 1966 (edit distance, for OCR error discussion).
38. M. Kleppmann, Designing Data-Intensive Applications, O'Reilly, 2017 (atomic replace, idempotence).
39. jsdom — https://github.com/jsdom/jsdom
40. arXiv API user manual — https://info.arxiv.org/help/api/user-manual.html
41. ISO 19005-1:2005, PDF/A-1: Document management applications — Electronic document file format for long-term preservation.
42. Git project — https://git-scm.com/
43. MIT License — https://opensource.org/license/mit
44. GitHub, REST API for repository creation and contents.
45. D. E. Knuth, The TeXbook, Addison-Wesley, 1984.
46. C. A. R. Hoare, Proof of a program: FIND, Commun. ACM 14(1), 1971.

附录 A 记忆体与 OCR 的 API 动作

记忆体的 Host 半在已鉴权通道上暴露两条路由与一组动作，OCR 的 Host 半暴露一条路由与两个动作。表 2（正文）给出动作清单及其 v2 变化；这里补出每个动作的契约细节，因为局限修复的很大一部分体现在这些契约上。

`POST /api/superman/api`，请求体为 `{action, ...}`。

- `list` → `{items[], notes[], stats, root}`：`stats` 含 `{total, bytes, notes, categories{pdf,image,code,other}, evicted, orphanFiles}`，其中 `evicted` 与 `orphanFiles` 是 v2 新增的计数。首次 `list` 会触发一次 `reconcile`。
- `store` → `{item, evicted[]}`：入参为 `{name, mime?, base64, source?, origin?, text?}`。v2 的行为变化有三点：`evicted` 是新增字段，报告本次入库触发的淘汰；Host 半在 `text` 缺失且分类为 PDF 时自行抽取正文（上限 200 页／400 000 字符）；入库后必然执行一次压缩检查。
- `get` → `{item}`：返回条目元数据与正文预览。
- `delete` → `{removed: true|false}`：删除条目及其文件，或删除一条记录。
- `note` → `{note}`：写入或更新一条记录（`kind` 为 `task` 或 `conversation`），用于任务记录与可还原会话。
- `recall` → `{matches[], digest}`：入参 `{q, limit?}`。v2 的响应形状未变（这是兼容性的关键），内部打分由加法打分改为 BM25；`limit` 默认 6、钳制在 1–20。
- `prune` → `{removed, freedBytes, items, bytes}`：v2 新增，按压缩策略立即执行一次。

`GET /api/superman/file?id=<id>` → 原始字节，`content-type` 取条目的媒体类型；未知 id 返回 404。v2 的新用途是历史图片取回原始字节（L4）。

`POST /api/superman-ocr/api`，**请求体为** `{action, ...}`。

- `health` → `{ready, bin, compiled, version}`：`ready` 表示当前有一个可执行的二进制；`compiled` 只在**本进程**编译过时为真（复用一个已缓存的产物不算）；`version` 来自 `--version` 输出，取不到时回落到 `ocr-pdf 1.0.0`。
- `ocr` → `{text, pages, characters}`：入参 `{base64, name?, maxPages?}`。`text` 是被识别文本（截断到 400 000 字符），`pages` 由输出中 `## page N` 标记的个数统计，`characters` 是 `text` 的长度。失败以稳定错误码返回：`bad-request`（400）、`too-large`（413）、`aborted`（499）、`ocr-timeout`（504）、`ocr-failed`（502）、`compile-failed`（500 系列）、`internal`（500）。

附录 B 验证套件清单与每套检查驱动的对象

六套检查的规模与驱动对象见表 6（正文）；这里逐套说明每个检查族真正驱动了什么，因为「检查数」本身不是覆盖面的度量。

`chat-mode-check.mjs`（**139 项，823 行**）。检查族：(a) 路由注册与请求校验，驱动真实的路由处理函数并对每种失败码断言状态与错误码；(b) SSE 分帧与重组，驱动真实的帧写入与页面读端，覆盖跨块边界的分帧；(c) 多轮历史与配额裁剪，驱动真实的 `localStorage` 替身与去抖写入；(d) 附件内联，覆盖网页去标签、Mathematica 笔记本拍平、文本编码识别与二进制跳过；(e) 图片转附件，驱动 base64 校验与附件保存替身；(f) 记忆体联动，覆盖上传入库、问题带摘要、轮次镜像与开关；(g) v2 新增：可还原载荷的写入与超长丢弃、空 origin 的恢复、新旧副本的合并规则、损坏载荷的跳过；(h) v2 新增：带 `memoryId` 的图片取回与 404 回退；(i) v2 新增：PDF 抽取为空时对 OCR 路由的调用与失败时的降级；(j) 失败路径与面板渲染，驱动桩 React 下的组件树断言。

`chat-mode-format-check.mjs`（**59 项，263 行**）。驱动真实的 LaTeX→MathML 翻译器（16 条表达式逐条断言、禁止反斜杠泄漏）、Markdown 块结构、`$100` 不误判为公式、7 个编码探针、一份 GBK 笔记本的端到端读取与网页去标签。v2 未增删检查。

`superman-host-check.mjs`（**71 项，501 行**）。检查族：(a) 分类级联（魔术字节、MIME、扩展名、内容启发式）；(b) 图片尺寸从文件头解析；(c) 存储与索引、原子写、并发入库；(d) `get` 与 `file` 路由；(e) 记录去重与更新；(f) 回忆与摘要；(g) 删除；(h) Harness 附件捕获与轮次记录；(i) v2 新增：BM25 的稀有词与 IDF 断言；(j) v2 新增：`maxItems`／`maxBytes` 上限、最旧优先淘汰、文件删除、字节上限保持、`prune` 计数、启动对账、索引重写与计数；(k) v2 新增：两端 PDF 抽取器对真实文件与合成文件的对等比较。其中的真实文件对等检查需要 `papers/` 目录下有 PDF；缺少语料时该检查无法通过（见附录 E）。

`superman-check.mjs`（**57 项，348 行**）。驱动记忆体面板：入口与次序、大脑图标、字典键集一致、六个分类页签渲染、卡片与预览、上传与删除请求、提示卡注册与去重、守望者基线、错误与空态降级。v2 未增删检查。

`arxiv-smoke.mjs`（**52 项，450 行**）。驱动 arXiv bundle：Host 检索路由的分支与限速、Atom 解析与渲染、复制 BibTeX，以及收藏到记忆体的三条路径（存 PDF、降级为记录、失败）。v2 未增删检查。

`superman-ocr-check.mjs`（**71 项，407 行**）。v2 新增。驱动 OCR 路由与信封、参数校验（缺 `base64`、空载荷、非 PDF、超大载荷、非 JSON 请求体、未知动作）、二进制解析与缓存、**编译只发生一次**（并发调用共享同一个 Promise）、编译失败的错误码、超时、临时文件的创建与清理、页数与字符数的统计、文本上限截断，以及在配置了 `config.bin` 时跳过编译的行为。它同时用真实的 `swiftc` 编译真实的 Swift 源文件，以验证编译命令与产物路径，但真实识别质量的验证不在这套检查里。

附录 C 语料与 OCR 测量

C.1 文本层抽取语料

语料由 14 份本机 PDF 组成，汇总如下：199 942 个字符、67 235 个汉字、138 页，替换字符与 C1 控制字符合计 0；5 份文档抽到文本层，9 份被如实报告为 `no-text`。逐文件的观测按语料特性可以分成四组。

长中文文档（2 份）。两份招生简章分别为 56 页与 54 页，抽出 90 938 与 89 764 个字符，其中 31 791 与 31 252 个汉字，约每页 1 600 余字符；中位耗时 71.8 ms 与 71.3 ms。它们是语料中唯一的两份高密度中文文档，也是「文字层抽取器在正常情况下表现良好」这一结论的唯一支撑，样本量因此很小。

中等长度文档（2 份）。一份 7 页合同抽出 9 031 个字符（3 789 个汉字），中位耗时 11.7 ms；一份 2 页文档抽出 786 个字符（394 个汉字），中位耗时 7.3 ms。

文字层极薄或与正文不符的文档（1 份）。一份 19 页招贴抽出 9 423 个字符，其中仅 9 个汉字，中位耗时 34.7 ms。字符不少而汉字近乎为零，说明其文字层并非读者眼中的正文；这一异常正是后来触发 OCR 分支的动机之一。

无文字层文档（9 份）。九份 Apple 系统生成的矢量图形 PDF，每份 0.2–0.8 ms，抽出的字符数为 0，全部被报告为 `no-text`。它们不是「空文件」：文件本身可以被 PDF 库打开并渲染，只是内容流中没有可抽取的文本算符，因此抽取器选择了拒绝而不是输出噪声。

关于**页数**的语义需要说明：9 份无文字层文件的页数为 0，而不是它们的真实页数。抽取器报告的页数是「重放过的页对象数量」，因此当它找不到任何含文本的页时给出 0；这意味着「页数」这一列在失败行上不可用于容量估计。关于**耗时**的语义同样需要说明：这些毫秒数是抽取器单次调用的中位值，包含解码与重放，不包含文件读取，也不包含任何网络或渲染；同一份文件在两次测量之间可能相差 20–30 ms（见 3.2 的威胁三）。关于**噪声**列：替换字符（U+FFFD）与 C1 控制字符的计数在全部 14 份文件上都是 0，这是实现选择的结果——抽取器宁可判定 `no-text` 或 `unreadable`，也不输出被污染的文本。

C.2 OCR 测量

抽取语料与 OCR 语料不是同一批文件：前者用于刻画文字层抽取器的行为，后者用于刻画 OCR 在无文字层页面上的恢复能力。OCR 的三个案例分别检验三种假设：文本层为 0 时 OCR 能否恢复文本（0 → 3 411 字符，979 个汉字）；文本层近乎为空时结果是否稳定（1 → 1 341 字符，1 072 个汉字）；以及文字层存在但内容与正文不符时 OCR 能否改善（9 423 字符／9 汉字 → 11 333 字符／66 汉字）。三例的替换字符计数均为 0。

我们没有给出字符错误率或字错误率，因为没有平行文本：三例都没有同一份文档的人工转录作为对照，因此只能报告字符数与汉字数的对照以及定性的误差形态（行内空格、形近字替换、标点规范化）。这是本工作的直接局限，也是「OCR 结果只作为可检索正文预览、不作为可引用来源」这一取舍的依据。OCR 的耗时形态与文字层抽取不同：它取决于引擎加载、渲染比例、页面尺寸与语言数量，秒级起步，因此本文不把它与毫秒级的文字层抽取直接比较；可比的量是它在一次调用内能完成的页数上限（默认 40 页、硬上限 200 页）。

关于**样本来源**，三例来自本机文件，按「无文字层或文字层与正文不符」这一特征挑选，而不是按标题挑选；挑选准则决定了它们能支持「这三类情形已被处理」，不能支持「扫描件整体已被解决」。

C.3 编码探测与公式电池（v1 数据，v2 沿用）

编码探测 7/7 正确：UTF-8、UTF-8 BOM、GBK（`a` 加 `C4 E3 BA C3`）、Big5（`A4 A4 A4 E5`）、无 BOM 的 UTF-16LE 被正确识别，PNG 文件头与随机高字节串被正确判为二进制。可信度分别为 1.769、1.0、2.333、3.0、1.8、0、0；GBK 与 Big5 的可信度高于 UTF-8 探针，说明「严格 UTF-8 失败后按候选集打分」靠分数而非枚举顺序作出判断，PNG 头与随机高字节串得 0 分，因此从未被内联。

16 条 LaTeX→MathML 表达式全部生成结构合法的 `<math>` 标记，且无反斜杠泄漏；电池覆盖分式、根式、上下标、大算符（求和、积分、环路积分）、矩阵与 `aligned`/`cases` 环境、核素记法、向量与重音、`\mathbb`/`\mathcal`、伸缩括号与带括号的单位矩阵；标记长度从 148 到 436 字符。这一组测量在 v2 中保持原值，因为相关代码未被修改。它同时提醒我们「449 项全绿」里包含 168 项在 v2 中未改动的能力的检查——其中 `chat-mode-format-check` 59 项、`superman-check` 57 项、`arxiv-smoke` 52 项——这些检查的价值在于**防止回归**，而不在于证明新能力；公式电池的 16 条断言的是标记合法与无反斜杠泄漏，不是与 TeX 的视觉一致性。

C.4 数据的一致性说明

有两处需要在数据层面说明清楚。

其一，文字层抽取的**汇总数字与逐文件数字来自不同的测量运行**。汇总值（199 942 个字符、67 235 个汉字、138 页）取自 v1 的参照测量，逐文件数据则存放在随稿的 `papers/data/metrics.json` 中；后者的字符数之和与前者不一致（同一类文件在两次运行中分别给出 199 942 与 227 522 个字符）。差异的可能来源有三：同一份文档在两次运行中被处理了不同的子集、抽取器的探测条件发生改变、或文档内容本身在这两次运行之间变化。我们在正文中使用参照测量作为可比量，并把逐文件数据保留为机器可读的事实记录，而不是反过来调整汇总值去迁就某一次运行。

其二，**同一实现、同一输入在同一运行内是确定性的**：检查套件对同一份字节调用两次抽取器会得到同一个字符串，对等检查正是建立在这条性质上。跨运行的差异只说明两次测量的输入或版本不同，不能用于推断某一版本的实现有缺陷。把这两点分开陈述，是因为混淆它们会得出相反结论：前者是数据溯源问题，后者是正确性问题。

C.5 独立的页面级复核

作为独立于项目的复核，我们在一台开发机上用同一套工具链（`swiftc` 编译出的 `tools/ocr-pdf.swift` + Vision）对一份与本项目语料无关的页面级栅格化 PDF 跑了一次完整路径，得到 1 666 个字符、其中 1 126 个汉字、0 个替换字符。这次复核的样本既不是本文三个案例中的任何一个，也不是用于汇总数字的语料，因此它只用于确认流水线在本机上端到端可用（编译、渲染、识别、输出四步都走通），其数值不进入正文的任何汇总。复核同时确认了一件事：`swiftc` 编译确实需要 `PDFKit` 一类系统框架，且把模块缓存指向可写目录后编译可重复成功——这正是 bundle 在编译时设置 `CLANG_MODULE_CACHE_PATH` 的原因。

附录 D 局限到修复到证据的登记表

登记表由 `papers/v2/data/metrics.json` 在排版时生成（正文表 4 即该表），因此本附录只给出该表每列的定义与判读方式，避免手写副本与生成表不一致。

- **编号**：局限的稳定标识 L1–L8。编号一旦分配不随版本改变，因此可以在检查、提交记录与文档之间交叉引用。
- **v1 局限**：v1 发布时写明的问题陈述，逐字保留其范围（例如 L1 说的是「只读文字层」，不是「抽取质量不高」）。
- **v2 状态**：三值之一——已关闭、部分关闭、未关闭。状态由证据决定，不由改动量决定；L1–L6 为已关闭，L7 为部分关闭，L8 为未关闭。
- **修复**：把修复实现的机制压缩到一行，写清它落在哪一半（客户端热重载 / Host 重启），因为生效方式对使用者是可见的差异。
- **证据（可复核）**：指向具体的检查名与图表编号，而不是「已测试」。表中的每一条证据都可以在发布树的对应套件里找到同名断言。

除 L1–L8 之外，登记表还包含一行不带编号的条目：v1 的去重规则（内容哈希 + 文件名）在 v2 中**未被单独修复**，只是不再导致无界增长。把它列在表里而不隐藏在正文里，是为了避免读者把「压缩策略存在」误读为「去重问题已解决」。同理，L7 与 L8 在登记表里保留了它们未完成的表述，而不是折叠成「未来工作」。

附录 E 复现命令与仓库布局

仓库把两个版本并列，便于逐条对照局限的修复：

```
dsh-superman-suite/
  v1/            # v1 ■■■■■■■■■■■■
    chat-mode/  superman/  arxiv-search/  tools/  docs/
  v2/            # ■■■■■■■■■■
    chat-mode/  superman/  superman-ocr/  arxiv-search/  tools/  docs/
  papers/        # ■■■■■■■■■■■■
```

复现全部离线检查：

```
cd dsh-superman-suite/v2
node tools/chat-mode-check.mjs && node tools/chat-mode-format-check.mjs \
  && node tools/superman-host-check.mjs && node tools/superman-check.mjs \
  && node tools/arxiv-smoke.mjs && node tools/superman-ocr-check.mjs
```

其中 `superman-host-check.mjs` 的「两端抽取器对等」检查需要工作目录下存在含 PDF 的 `papers/` 目录；缺少语料时该项无法通过，其余检查不受影响。复现语料测量与图表数据：

```
node tools/paper-metrics.mjs
MPLCONFIGDIR=/tmp/mpl python3 papers/tools/figures.py
python papers/tools/build_pdf.py      # ■■ reportlab
```

OCR 路径可单独复现：`swiftc -O tools/ocr-pdf.swift -o ocr-pdf -framework Vision -framework PDFKit -framework AppKit`，随后 `./ocr-pdf <■■.pdf> --max-pages 1 --scale 2`，输出为每页一段 `## page N` 标记加该页文本。DOM 工具链位于工作区的 `tools/dom/`（本地 `jsdom`、`react`、`react-dom`），入口为 `node tools/dom-check.mjs`；该套件当前不能通过，因此未随发布目录分发。